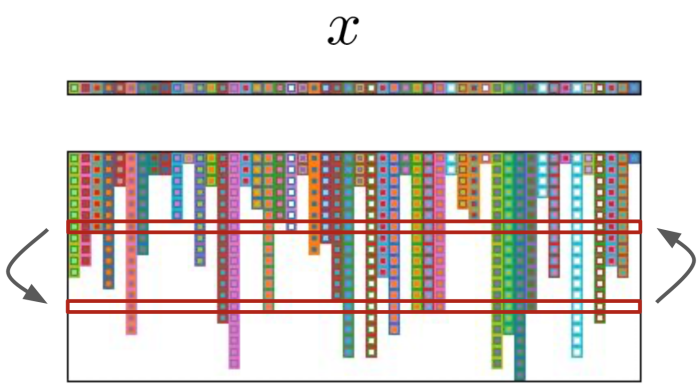
Deep Learning for NLP II

Diffusion Language Models



Learning goals

- Deep latent variable models
- Diffusion models as Markovian hierarchical VAEs
- Masked diffusion language models

DIFFUSION LLMS IN ACTION

Prompt: *What is a diffusion LM? Answer in one sentence.*

Response:

[MASK]	[MASK]	[MASK]	[MASK]	[MASK]	[MASK]	[MASK]
[MASK]	[MASK]	[MASK]	[MASK]	[MASK]	[MASK]	[MASK]
[MASK]	[MASK]	[MASK]	[MASK]	[MASK]	[MASK]	[MASK]

DIFFUSION LLMS IN ACTION

Prompt: *What is a diffusion LM? Answer in one sentence.*

Response:

[MASK]	diffusion	LM	[MASK]	[MASK]	[MASK]	[MASK]
[MASK]	[MASK]	[MASK]	[MASK]	[MASK]	[MASK]	[MASK]
[MASK]	[MASK]	[MASK]	[MASK]	.	<eos>	<eos>

DIFFUSION LLMS IN ACTION

Prompt: *What is a diffusion LM? Answer in one sentence.*

Response:

A	diffusion	LM	[MASK]	[MASK]	[MASK]	masked
[MASK]	[MASK]	[MASK]	[MASK]	tokens	[MASK]	[MASK]
[MASK]	[MASK]	[MASK]	step	.	<eos>	<eos>

DIFFUSION LLMS IN ACTION

Prompt: *What is a diffusion LM? Answer in one sentence.*

Response:

A	diffusion	LM	starts	from	[MASK]	masked
text	[MASK]	unmasks	[MASK]	tokens	[MASK]	[MASK]
[MASK]	[MASK]	[MASK]	step	.	<eos>	<eos>

DIFFUSION LLMS IN ACTION

Prompt: *What is a diffusion LM? Answer in one sentence.*

Response:

A	diffusion	LM	starts	from	[MASK]	masked
text	and	unmasks	[MASK]	tokens	in	parallel
[MASK]	step	[MASK]	step	.	<eos>	<eos>

DIFFUSION LLMS IN ACTION

Prompt: *What is a diffusion LM? Answer in one sentence.*

Response:

A	diffusion	LM	starts	from	[MASK]	masked
text	and	unmasks	[MASK]	tokens	in	parallel
,	step	by	step	.	<eos>	<eos>

DIFFUSION LLMS IN ACTION

Prompt: *What is a diffusion LM? Answer in one sentence.*

Response:

A	diffusion	LM	starts	from	fully	masked
text	and	unmasks	many	tokens	in	parallel
,	step	by	step	.	<eos>	<eos>

DIFFUSION LLMS IN ACTION

Prompt: *What is a diffusion LM? Answer in one sentence.*

Response:

A	diffusion	LM	starts	from	fully	masked
text	and	unmasks	many	tokens	in	parallel
,	step	by	step	.	<eos>	<eos>

What do you observe?

OUTLINE

Deep Latent Variable Models

Diffusion Models

Masked Diffusion LLMs

OUTLINE

Deep Latent Variable Models

Diffusion Models

Masked Diffusion LLMs

RECAP: UNIGRAM LATENT VARIABLE MODEL

- Observed variable **S** (string), latent variable **V** (segmentation)
- Prior over segmentations: $P(\mathbf{V} = \mathbf{v}; \phi) = \prod_{t=1}^{|\mathbf{v}|} \phi_{v_t}$
- Deterministic likelihood: $P(\mathbf{S} = \mathbf{s} \mid \mathbf{V} = \mathbf{v}) = \mathbb{1}[g(\mathbf{v}) = \mathbf{s}]$
- Marginal (pushforward): $P(\mathbf{S} = \mathbf{s}; \phi) = \sum_{\mathbf{v} \in \mathcal{T}_{\mathcal{V}}(\mathbf{s})} P(\mathbf{V} = \mathbf{v}; \phi)$
- Posterior for $\mathbf{v} \in \mathcal{T}_{\mathcal{V}}(\mathbf{s})$ (Bayes' rule):

$$P(\mathbf{V} = \mathbf{v} \mid \mathbf{S} = \mathbf{s}; \phi) = \frac{P(\mathbf{V} = \mathbf{v}; \phi)}{P(\mathbf{S} = \mathbf{s}; \phi)}$$

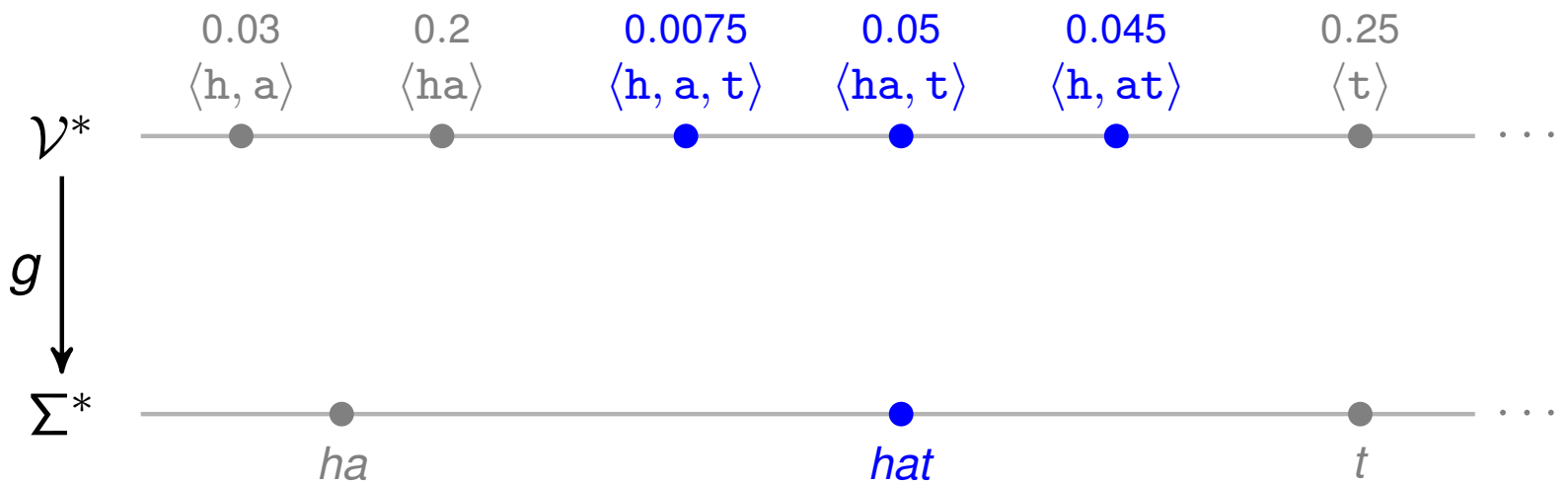
RECAP: EM FOR UNIGRAM

- Goal: maximize the observed data log-likelihood $\log P(\mathbf{S} = \mathbf{s}; \phi)$
- As a proxy, we maximize the expected complete data log-likelihood $Q(\phi; \phi^{(n)}) = \sum_{\mathbf{v} \in \mathcal{T}_{\mathcal{V}}(\mathbf{s})} P(\mathbf{V} = \mathbf{v} \mid \mathbf{S} = \mathbf{s}; \phi^{(n)}) \log P(\mathbf{V} = \mathbf{v}; \phi)$
- $\log P(\mathbf{S} = \mathbf{s}; \phi)$ is related to $Q(\phi; \phi^{(n)})$ as follows:

$$\log P(\mathbf{S} = \mathbf{s}; \phi) = \underbrace{Q(\phi; \phi^{(n)}) + H(\mathbf{V} \mid \mathbf{S} = \mathbf{s}; \phi^{(n)})}_{\text{evidence lower bound (ELBO)}} + \underbrace{\mathrm{KL}\left(P(\mathbf{V} \mid \mathbf{S} = \mathbf{s}; \phi^{(n)}) \,\middle\|\, P(\mathbf{V} \mid \mathbf{S} = \mathbf{s}; \phi)\right)}_{\geq 0;\ 0 \text{ at } \phi = \phi^{(n)}}$$

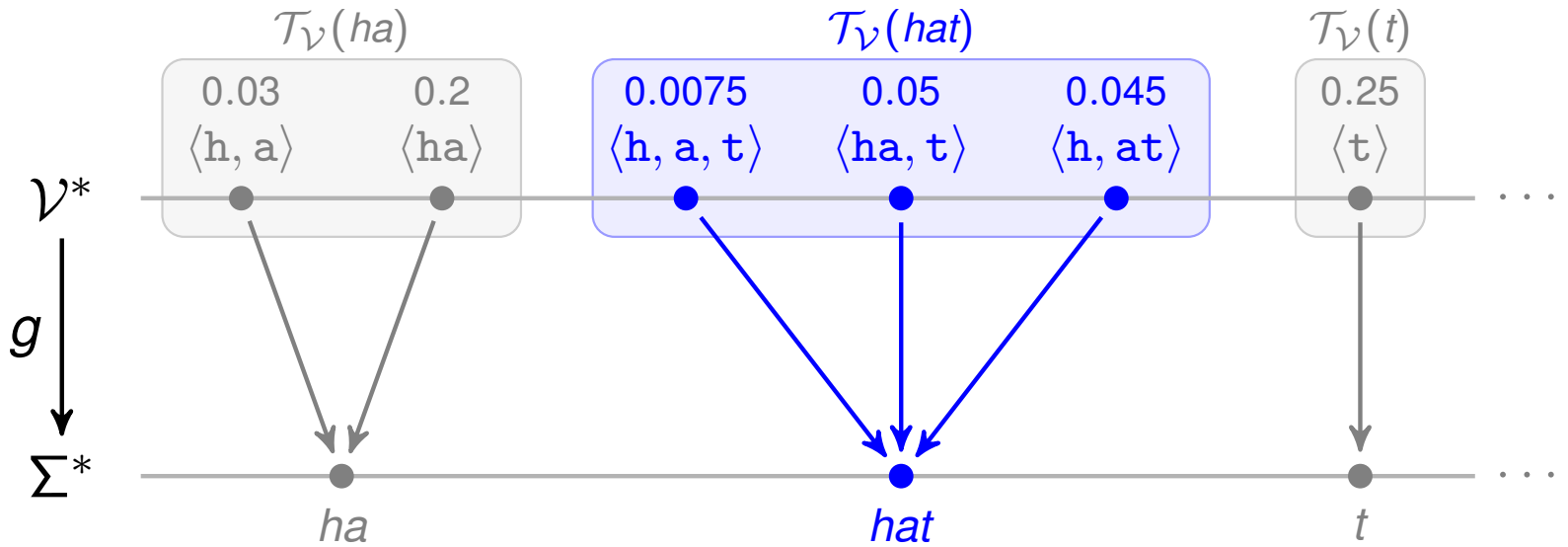
- **E-step**: compute the exact posterior $P(\mathbf{V} \mid \mathbf{S} = \mathbf{s}; \phi^{(n)})$ and expected counts with forward-backward
- **M-step**: maximize $Q(\phi; \phi^{(n)})$ by normalizing the expected counts

RECAP: VISUALIZED EXAMPLE



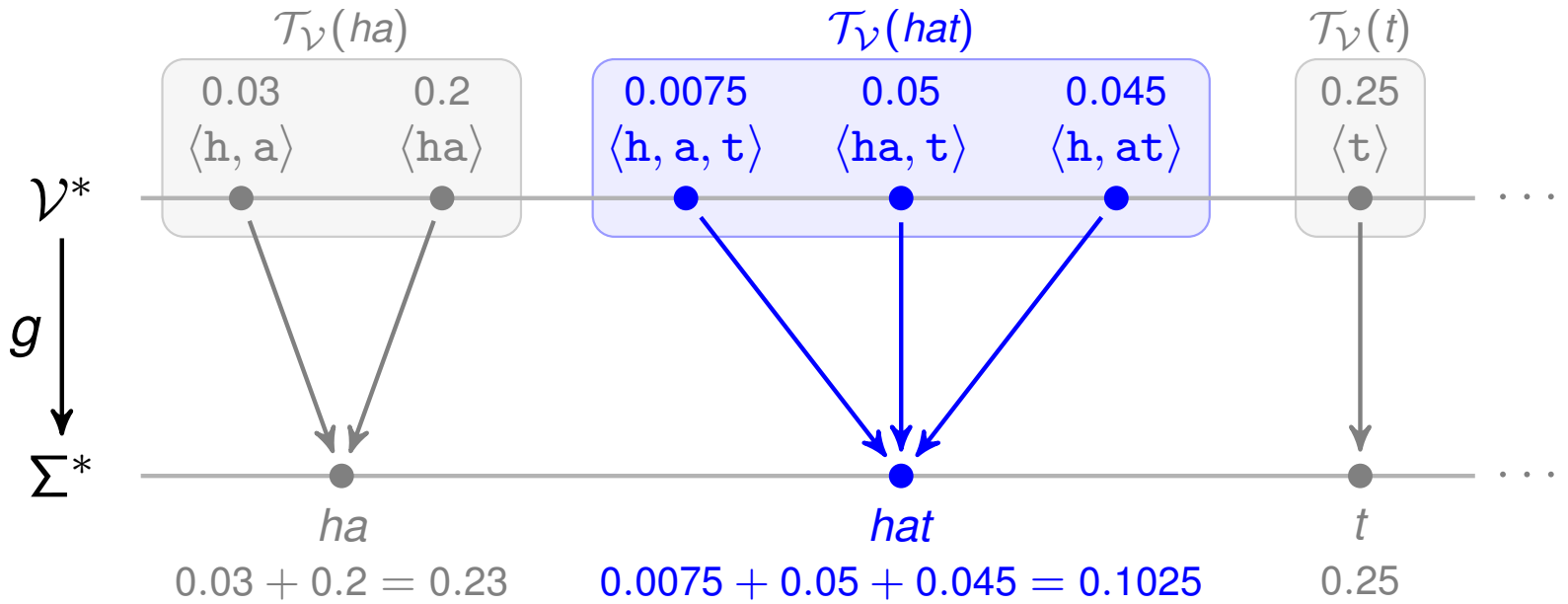
- Prior $P(\mathbf{V} = \mathbf{v}; \phi)$ on top of each token sequence

RECAP: VISUALIZED EXAMPLE



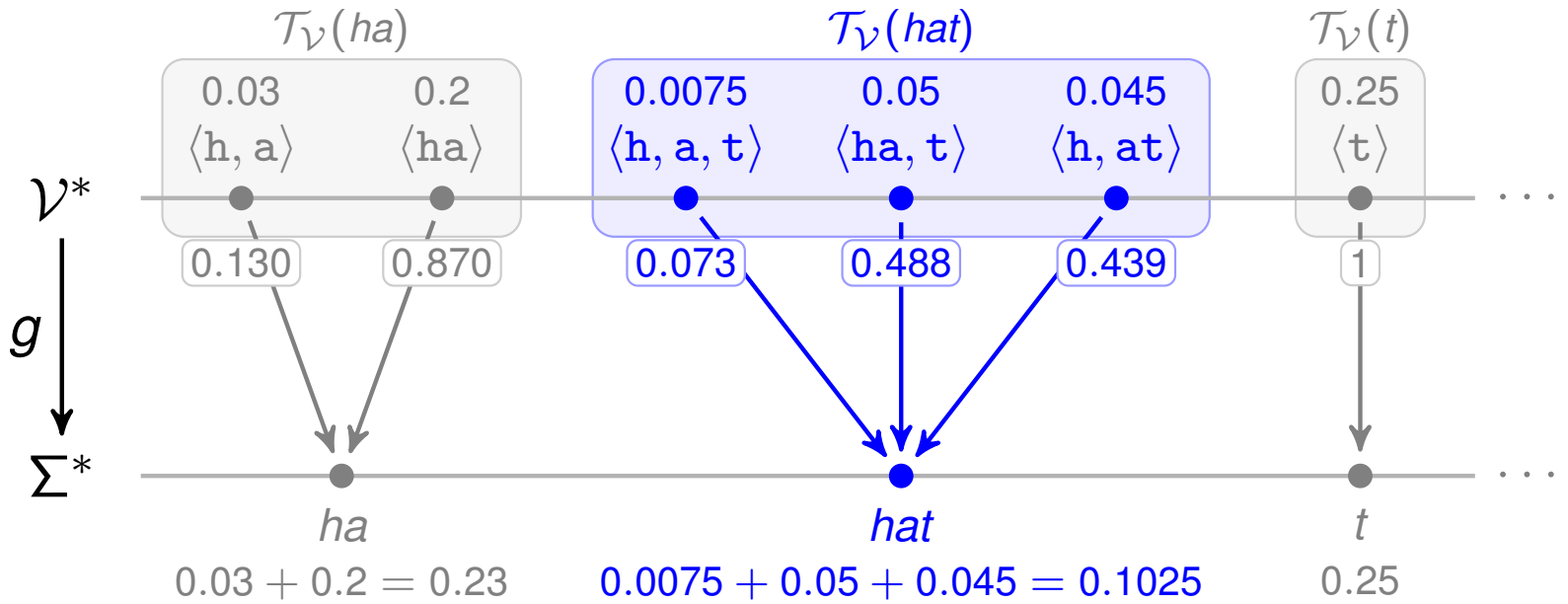
- Prior $P(\mathbf{V} = \mathbf{v}; \phi)$ on top of each token sequence
- g maps each token sequence to one string

RECAP: VISUALIZED EXAMPLE



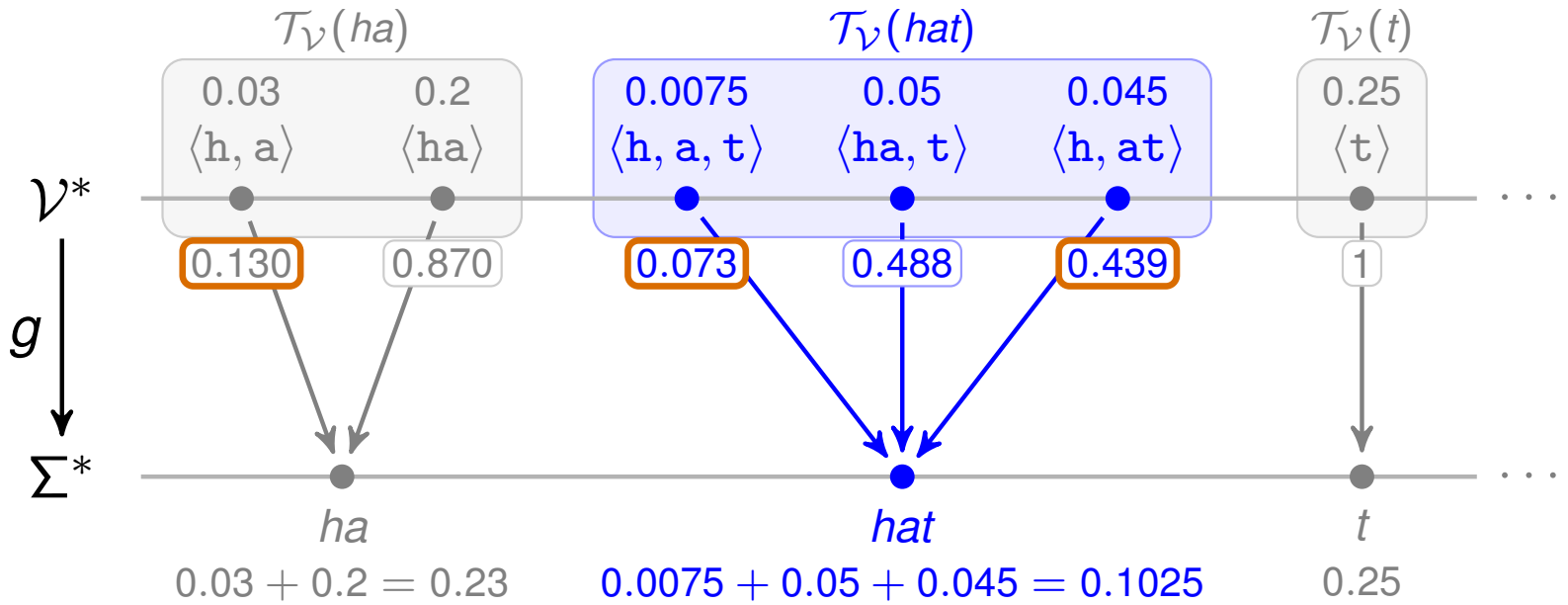
- Prior $P(\mathbf{V} = \mathbf{v}; \phi)$ on top of each token sequence
- g maps each token sequence to one string
- Marginal: each string collects the mass of its preimage

RECAP: VISUALIZED EXAMPLE



- Prior $P(\mathbf{V} = \mathbf{v}; \phi)$ on top of each token sequence
- g maps each token sequence to one string
- Marginal: each string collects the mass of its preimage
- Posterior: renormalize the prior within each preimage

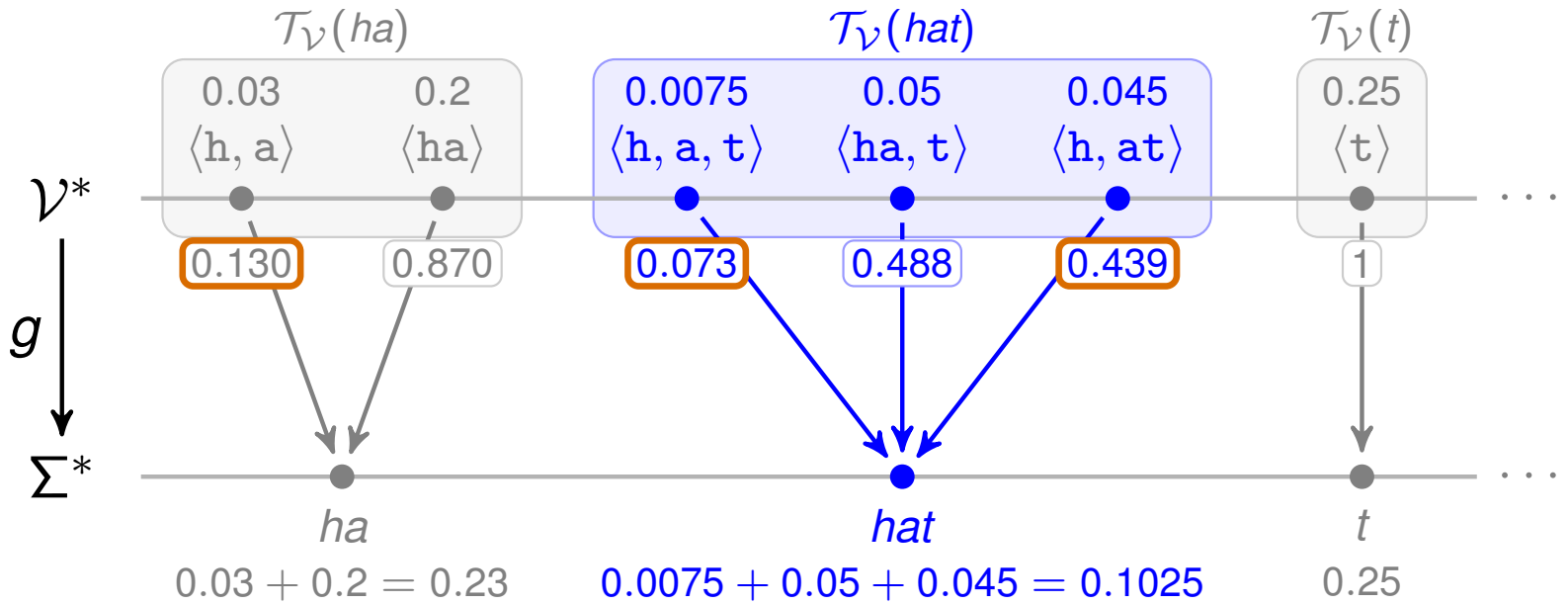
RECAP: VISUALIZED EXAMPLE



$\mathcal{C} = \{ha, hat, t\}$	h	a	t	ha	at	sum
$\phi_v^{(n)}$	0.3	0.1	0.25	0.2	0.15	1
$\hat{c}_v(\mathcal{C}; \phi^{(n)})$	**0.643**	0.204	1.561	1.357	0.439	4.204
$\phi_v^{(n+1)}$						

- **E-step**: $\hat{c}_{\mathtt{h}} = 0.130 + 0.073 + 0.439 \approx 0.643$

RECAP: VISUALIZED EXAMPLE



$\mathcal{C} = \{ha, hat, t\}$	h	a	t	ha	at	sum
$\phi_v^{(n)}$	0.3	0.1	0.25	0.2	0.15	1
$\hat{c}_v(\mathcal{C}; \phi^{(n)})$	**0.643**	0.204	1.561	1.357	0.439	4.204
$\phi_v^{(n+1)}$	**0.153**	0.048	0.371	0.323	0.104	1

- **E-step**: $\hat{c}_\mathtt{h} = 0.130 + 0.073 + 0.439 \approx 0.643$
- **M-step**: $\phi_\mathtt{h}^{(n+1)} = 0.643/4.204 \approx 0.153$

NOTATION

- Shorthand:
  - Observed variable $\mathbf{x}$, latent variable $\mathbf{z}$, model parameters θ, ϕ
  - $p_\theta(\mathbf{x}) := P(\mathbf{X} = \mathbf{x}; \theta)$, and analogously $p_\theta(\mathbf{z} \mid \mathbf{x})$, etc.
  - p: probability mass function (discrete) or density (continuous)
- Unigram in this notation:

$$\mathbf{x} = \mathbf{s}, \quad \mathbf{z} = \mathbf{v}, \quad \theta = \phi, \quad p_\theta(\mathbf{x} \mid \mathbf{z}) = \mathbb{1}[g(\mathbf{z}) = \mathbf{x}]$$

DEEP LATENT VARIABLE MODEL BASICS

- In **deep latent variable models**, **z** captures abstract properties of **x** (e.g., meaning), and a neural network maps **z** to **x**
- Generative story:

$$\mathbf{z} \sim p(\mathbf{z}) \qquad \text{(prior: e.g., } \mathcal{N}(\mathbf{0}, \mathbf{I})\text{)}$$
$$\mathbf{x} \sim p_\theta(\mathbf{x} \mid \mathbf{z}) \qquad \text{(likelihood: neural network)}$$

- Marginal likelihood: $p_\theta(\mathbf{x}) = \int p(\mathbf{z})\, p_\theta(\mathbf{x} \mid \mathbf{z})\, d\mathbf{z}$
- Same recipe as Unigram (model observed data by marginalizing out the latent), but with much more flexible components
- Problems:
  - High-dimensional integral over a neural network is intractable
  - Posterior $p_\theta(\mathbf{z} \mid \mathbf{x})$ requires $p_\theta(\mathbf{x})$, so it is also intractable $\rightarrow$ we cannot apply EM like for Unigram

ELBO FOR DEEP LATENT VARIABLE MODELS

- Idea: approximate the intractable posterior by a distribution $q_\phi(\mathbf{z} \mid \mathbf{x})$ from a tractable family (e.g., Gaussians)
- For *any* such q_ϕ, we can write (using $\int q_\phi(\mathbf{z} \mid \mathbf{x})\, d\mathbf{z} = 1$):

$$
\begin{aligned}
\log p_\theta(\mathbf{x}) &= \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\big[\log p_\theta(\mathbf{x})\big] = \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\left[\log \frac{p_\theta(\mathbf{x}, \mathbf{z})}{p_\theta(\mathbf{z} \mid \mathbf{x})}\right] \\
&= \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\left[\log \frac{p_\theta(\mathbf{x}, \mathbf{z})\, q_\phi(\mathbf{z} \mid \mathbf{x})}{p_\theta(\mathbf{z} \mid \mathbf{x})\, q_\phi(\mathbf{z} \mid \mathbf{x})}\right] \\
&= \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\left[\log \frac{p_\theta(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z} \mid \mathbf{x})}\right]}_{\mathrm{ELBO}(\phi,\theta;\mathbf{x})} + \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\left[\log \frac{q_\phi(\mathbf{z} \mid \mathbf{x})}{p_\theta(\mathbf{z} \mid \mathbf{x})}\right]}_{\mathrm{KL}(q_\phi(\mathbf{z}|\mathbf{x}) \,\|\, p_\theta(\mathbf{z}|\mathbf{x}))}
\end{aligned}
$$

ELBO FOR DEEP LATENT VARIABLE MODELS

- Idea: approximate the intractable posterior by a distribution $q_\phi(\mathbf{z} \mid \mathbf{x})$ from a tractable family (e.g., Gaussians)
- For *any* such q_ϕ, we can write (using $\int q_\phi(\mathbf{z} \mid \mathbf{x})\, d\mathbf{z} = 1$):

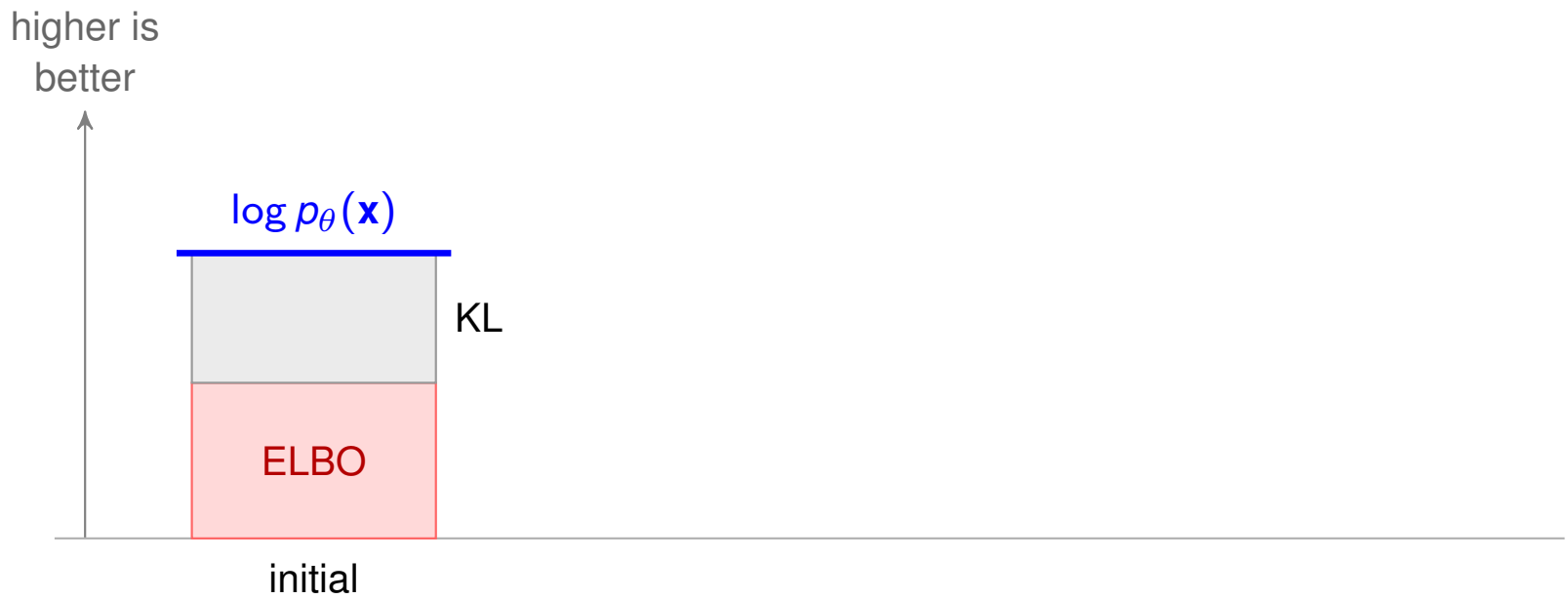
$$\begin{aligned}
\log p_\theta(\mathbf{x}) &= \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\big[\log p_\theta(\mathbf{x})\big] = \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\left[\log \frac{p_\theta(\mathbf{x}, \mathbf{z})}{p_\theta(\mathbf{z} \mid \mathbf{x})}\right] \\
&= \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\left[\log \frac{p_\theta(\mathbf{x}, \mathbf{z})\, q_\phi(\mathbf{z} \mid \mathbf{x})}{p_\theta(\mathbf{z} \mid \mathbf{x})\, q_\phi(\mathbf{z} \mid \mathbf{x})}\right] \\
&= \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\left[\log \frac{p_\theta(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z} \mid \mathbf{x})}\right]}_{\mathrm{ELBO}(\phi,\theta;\mathbf{x})} + \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\left[\log \frac{q_\phi(\mathbf{z} \mid \mathbf{x})}{p_\theta(\mathbf{z} \mid \mathbf{x})}\right]}_{\mathrm{KL}(q_\phi(\mathbf{z}|\mathbf{x}) \,\|\, p_\theta(\mathbf{z}|\mathbf{x}))}
\end{aligned}$$

- Connection to last week: splitting the logarithm gives

$$\mathrm{ELBO}(\phi, \theta; \mathbf{x}) = \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\big[\log p_\theta(\mathbf{x}, \mathbf{z})\big] + H\big[q_\phi(\mathbf{z} \mid \mathbf{x})\big]$$

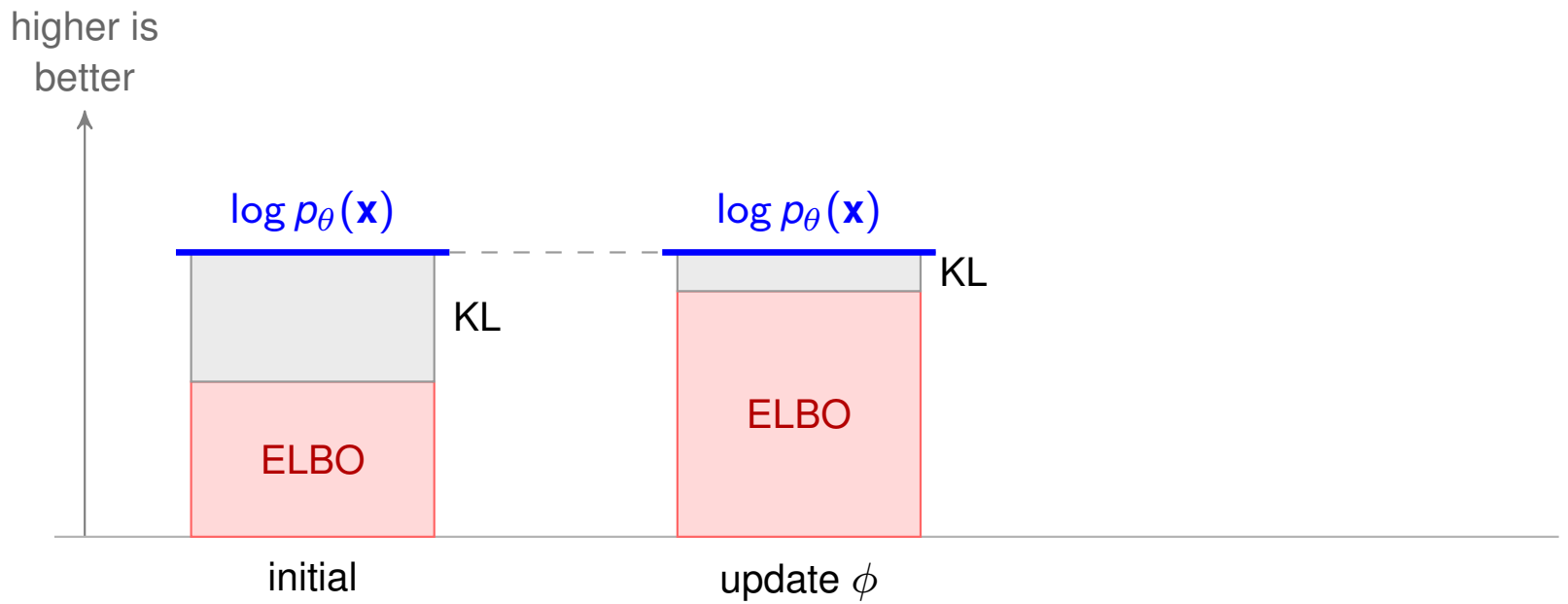
- If q_ϕ is the Unigram posterior, the first term is exactly Q

WHY MAXIMIZE THE ELBO?



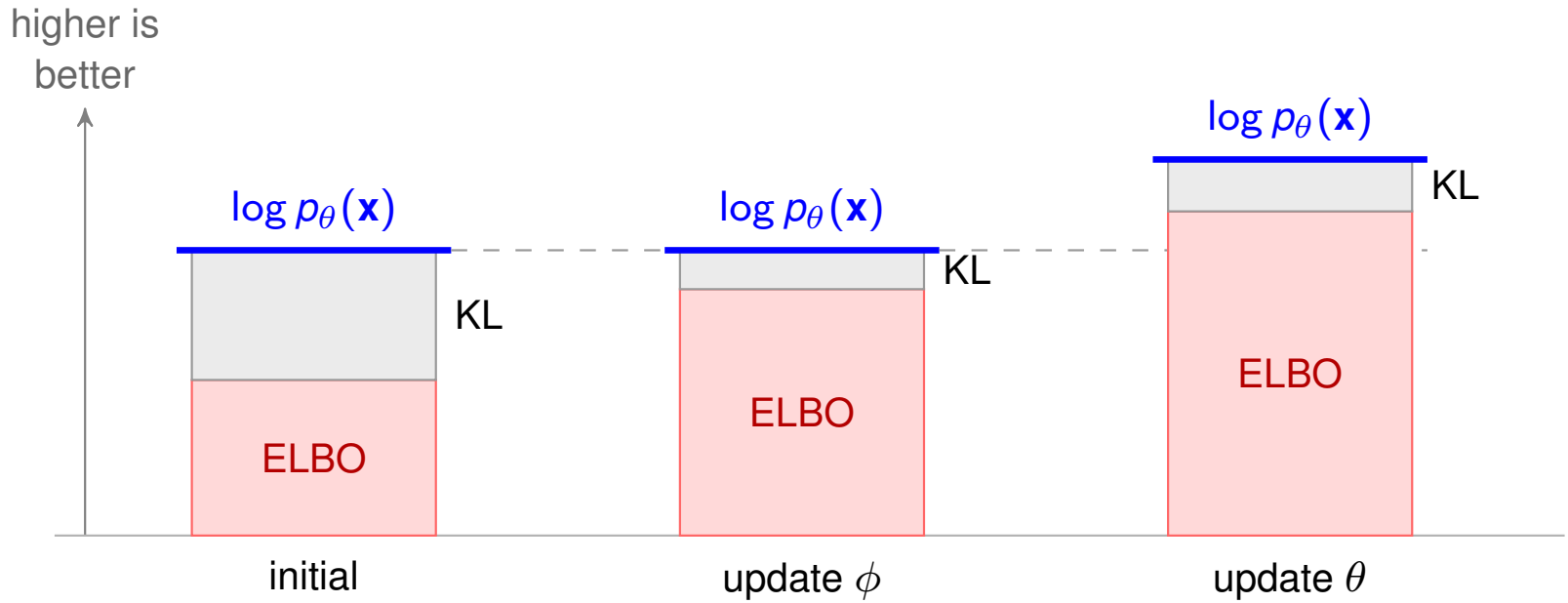
- ELBO and KL always add up to $\log p_\theta(\mathbf{x})$

WHY MAXIMIZE THE ELBO?



- ELBO and KL always add up to $\log p_\theta(\mathbf{x})$
- $\log p_\theta(\mathbf{x})$ does not depend on ϕ: increasing the ELBO w.r.t. ϕ must decrease the KL, so q_ϕ moves toward the **true posterior**

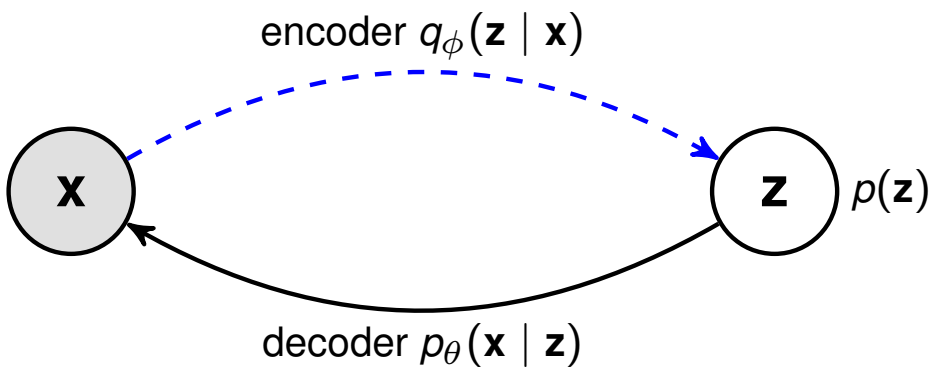
WHY MAXIMIZE THE ELBO?



- ELBO and KL always add up to $\log p_\theta(\mathbf{x})$
- $\log p_\theta(\mathbf{x})$ does not depend on ϕ: increasing the ELBO w.r.t. ϕ must decrease the KL, so q_ϕ moves toward the **true posterior**
- Increasing the ELBO w.r.t. θ pushes up a lower bound on $\log p_\theta(\mathbf{x})$, so the model improves

VARIATIONAL AUTOENCODERS (VAES)

- Introduced independently by Kingma and Welling (2014) ▸ Kingma and Welling (2014)[3] and Rezende et al. (2014) ▸ Rezende et al. (2014)[5]
- Maximize the ELBO jointly over ϕ and θ with gradient ascent
  - **Encoder** $q_\phi(\mathbf{z} \mid \mathbf{x})$: maps $\mathbf{x}$ to a distribution over latents
  - **Decoder** $p_\theta(\mathbf{x} \mid \mathbf{z})$: maps latents back to a distribution over observations
- **Amortized inference**: VAEs learn a *single* network that outputs $q_\phi(\mathbf{z} \mid \mathbf{x})$ for any $\mathbf{x}$



VAE OBJECTIVE

- Plugging $p_\theta(\mathbf{x}, \mathbf{z}) = p_\theta(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})$ into $\text{ELBO}(\phi, \theta; \mathbf{x})$:

$$
\begin{aligned}
\text{ELBO}(\phi, \theta; \mathbf{x}) &= \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} \left[\log \frac{p_\theta(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})}{q_\phi(\mathbf{z} \mid \mathbf{x})} \right] \\
&= \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} \big[\log p_\theta(\mathbf{x} \mid \mathbf{z}) \big] + \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} \left[\log \frac{p(\mathbf{z})}{q_\phi(\mathbf{z} \mid \mathbf{x})} \right] \\
&= \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} \big[\log p_\theta(\mathbf{x} \mid \mathbf{z}) \big]}_{\text{reconstruction term}} - \underbrace{\text{KL}\left(q_\phi(\mathbf{z} \mid \mathbf{x}) \,\|\, p(\mathbf{z}) \right)}_{\text{prior matching term}}
\end{aligned}
$$

- **Reconstruction term**: latents sampled from the encoder should allow the decoder to regenerate **x**
- **Prior matching term**: the encoder should not stray too far from the prior (why?)

STANDARD CHOICES FOR VAES

- Prior: $p(\mathbf{z}) = \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$
- Encoder: Gaussian with diagonal covariance

$$q_\phi(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}\big(\mathbf{z}; \boldsymbol{\mu}_\phi(\mathbf{x}), \mathrm{diag}(\boldsymbol{\sigma}^2_\phi(\mathbf{x}))\big)$$

- These choices make prior matching term computable analytically
- Reconstruction term: Monte Carlo estimate with $\mathbf{z}^{(l)} \sim q_\phi(\mathbf{z} \mid \mathbf{x})$

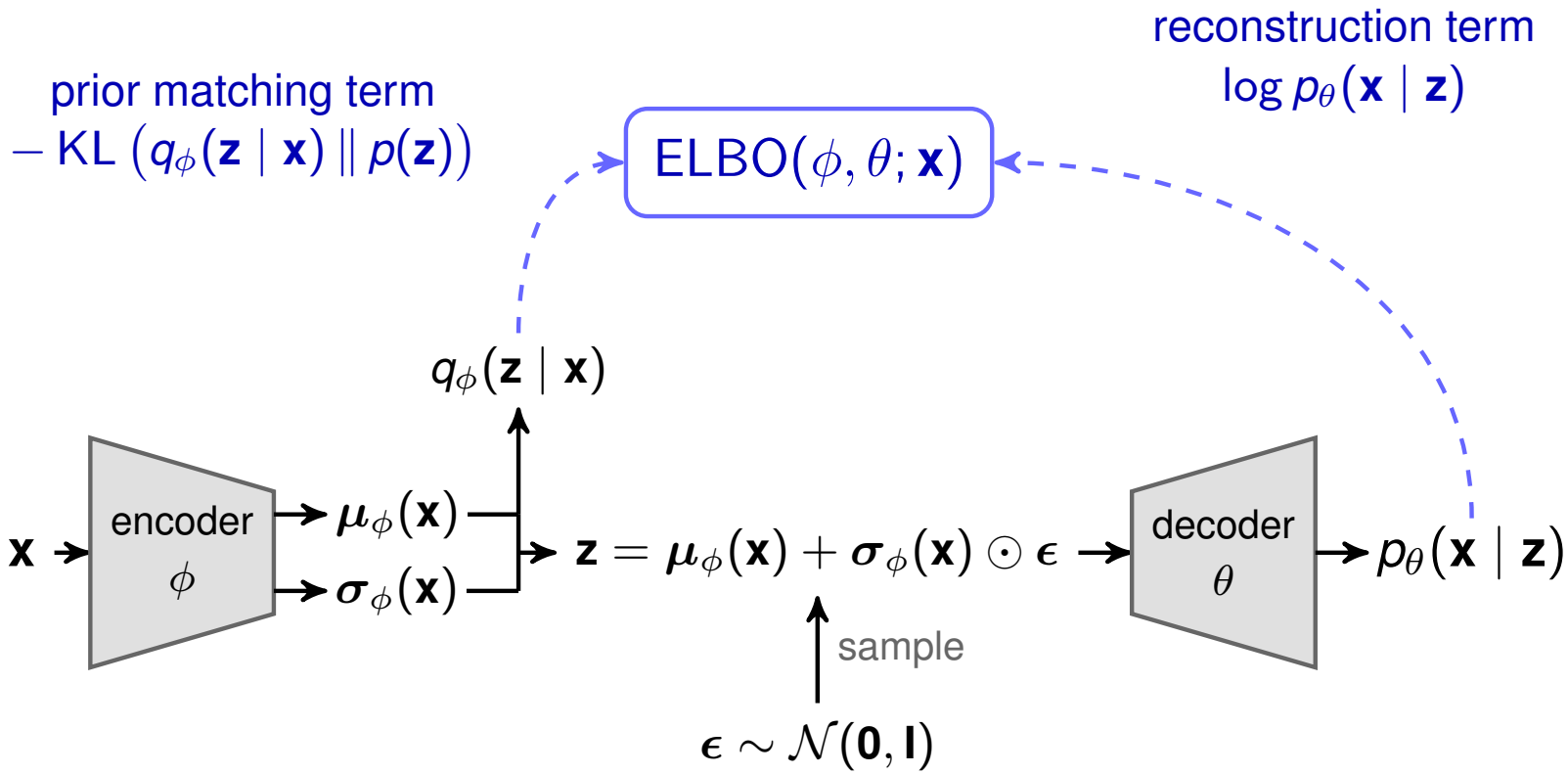
$$\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\big[\log p_\theta(\mathbf{x} \mid \mathbf{z})\big] \approx \frac{1}{L}\sum_{l=1}^{L} \log p_\theta(\mathbf{x} \mid \mathbf{z}^{(l)})$$

- **Reparameterization trick**: since we cannot backpropagate into ϕ through a sampling step, we write the sample as a deterministic function of ϕ and parameter-free noise:

$$\mathbf{z} = \mathbf{z}_\phi(\boldsymbol{\epsilon}) := \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

- Randomness now sits in $\boldsymbol{\epsilon}$, so gradients flow through $\boldsymbol{\mu}_\phi$ and $\boldsymbol{\sigma}_\phi$

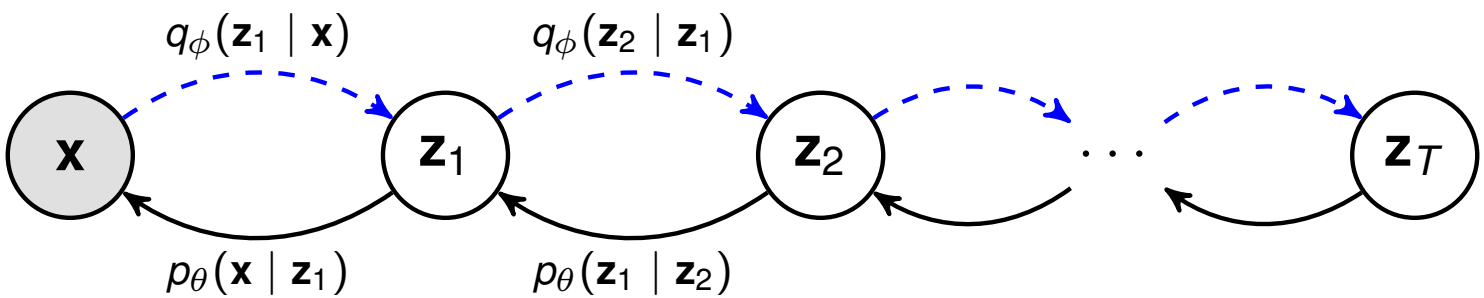
PUTTING THE VAE TOGETHER



- Training: encode, sample via reparameterization, decode, and maximize the ELBO w.r.t. ϕ and θ (SGD)
- Generation: drop the encoder, sample $\mathbf{z} \sim p(\mathbf{z})$, and decode

HIERARCHICAL VAES

- Idea: several levels of latents $\mathbf{z}_1, \ldots, \mathbf{z}_T$, with higher levels encoding more abstract properties ▸ Vahdat and Kautz (2020)[7]
- In a general hierarchical VAE (HVAE), each latent may condition on all previous latents
- **Markovian HVAE**: the process is a Markov chain, so each latent depends only on the one directly before it
- Intuitively, a Markovian HVAE is a stack of VAEs



MARKOVIAN HVAES

- Joint distribution ($\mathbf{z}_T \to \cdots \to \mathbf{z}_1 \to \mathbf{x}$):

$$p_\theta(\mathbf{x}, \mathbf{z}_{1:T}) = p(\mathbf{z}_T)\, p_\theta(\mathbf{x} \mid \mathbf{z}_1) \prod_{t=2}^{T} p_\theta(\mathbf{z}_{t-1} \mid \mathbf{z}_t)$$

- Approximate posterior ($\mathbf{x} \to \mathbf{z}_1 \to \cdots \to \mathbf{z}_T$):

$$q_\phi(\mathbf{z}_{1:T} \mid \mathbf{x}) = q_\phi(\mathbf{z}_1 \mid \mathbf{x}) \prod_{t=2}^{T} q_\phi(\mathbf{z}_t \mid \mathbf{z}_{t-1})$$

- Decoder and encoder consist of T learned conditional distributions
- Top-level prior $p(\mathbf{z}_T)$ is fixed (e.g., $\mathcal{N}(\mathbf{0}, \mathbf{I})$)

ELBO FOR MARKOVIAN HVAES

- Same derivation as before ($\mathbb{E}_{q_\phi}$ short for $\mathbb{E}_{q_\phi(\mathbf{z}_{1:T}|\mathbf{x})}$):

$$\begin{aligned}
\log p_\theta(\mathbf{x}) &= \log \int p_\theta(\mathbf{x}, \mathbf{z}_{1:T})\, d\mathbf{z}_{1:T} \\
&\geq \mathbb{E}_{q_\phi}\left[\log \frac{p_\theta(\mathbf{x}, \mathbf{z}_{1:T})}{q_\phi(\mathbf{z}_{1:T} \mid \mathbf{x})}\right] \\
&= \mathbb{E}_{q_\phi}\left[\log \frac{p(\mathbf{z}_T)\, p_\theta(\mathbf{x} \mid \mathbf{z}_1) \prod_{t=2}^{T} p_\theta(\mathbf{z}_{t-1} \mid \mathbf{z}_t)}{q_\phi(\mathbf{z}_1 \mid \mathbf{x}) \prod_{t=2}^{T} q_\phi(\mathbf{z}_t \mid \mathbf{z}_{t-1})}\right]
\end{aligned}$$

- For $T = 1$, we recover the VAE
- Challenges:
  - $2T$ conditional distributions have to be learned jointly
  - Deep hierarchies are hard to train in practice (e.g., higher-level latents often end up unused)

OUTLINE

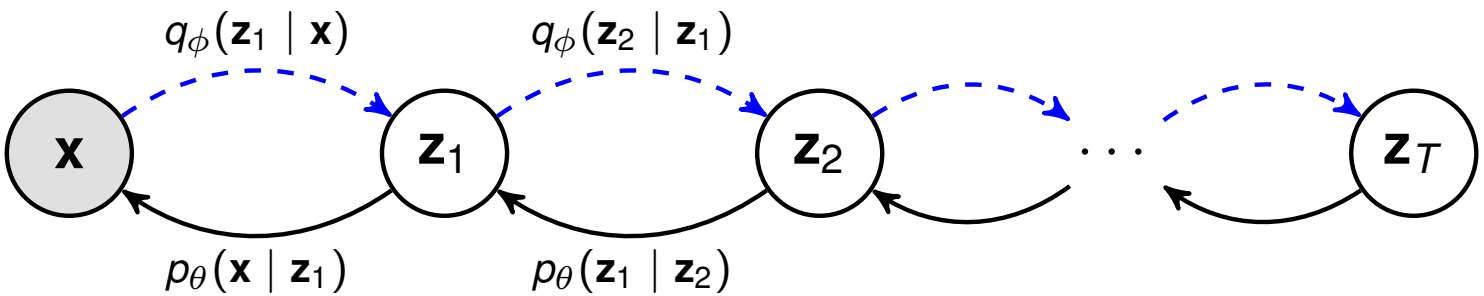
Deep Latent Variable Models

Diffusion Models

Masked Diffusion LLMs

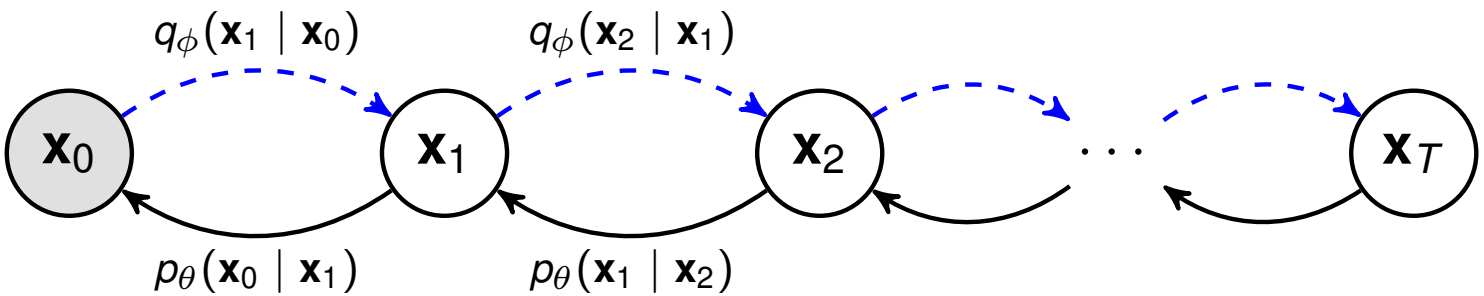
DIFFUSION MODELS AS MARKOVIAN HVAES

- Diffusion models are Markovian HVAEs with three restrictions:



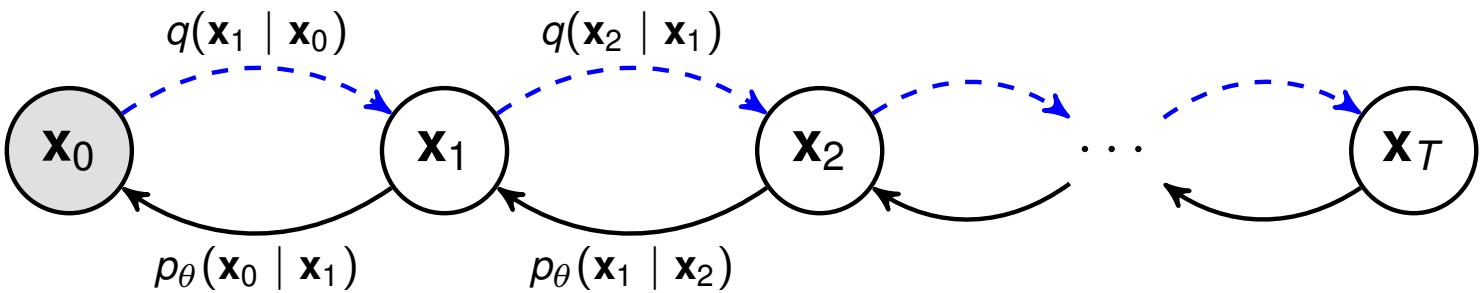
DIFFUSION MODELS AS MARKOVIAN HVAES

- Diffusion models are Markovian HVAEs with three restrictions:
  1. Latents have same dimensionality as data: $\mathbf{x}_0 := \mathbf{x}$, $\mathbf{x}_t := \mathbf{z}_t$



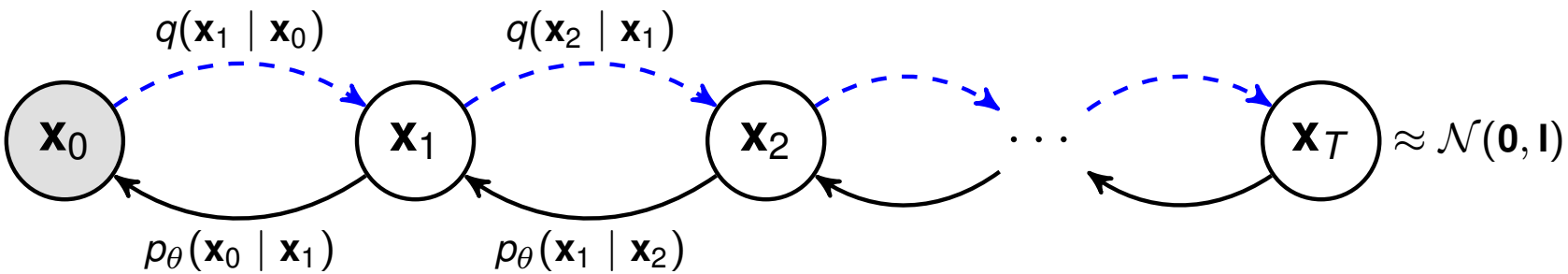
DIFFUSION MODELS AS MARKOVIAN HVAES

- Diffusion models are Markovian HVAEs with three restrictions:
  1. Latents have same dimensionality as data: $\mathbf{x}_0 := \mathbf{x}$, $\mathbf{x}_t := \mathbf{z}_t$
  2. The encoder is fixed: $q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$ gradually adds noise



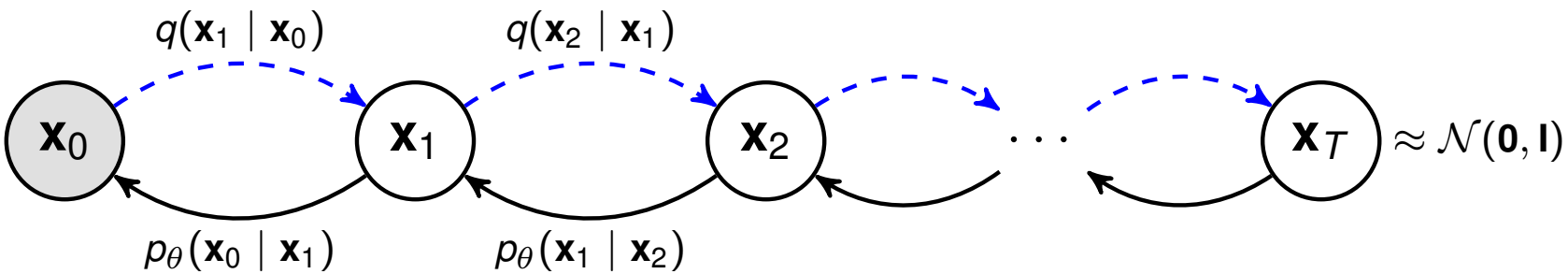
DIFFUSION MODELS AS MARKOVIAN HVAES

- Diffusion models are Markovian HVAEs with three restrictions:
  1. Latents have same dimensionality as data: $\mathbf{x}_0 := \mathbf{x}$, $\mathbf{x}_t := \mathbf{z}_t$
  2. The encoder is fixed: $q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$ gradually adds noise
  3. Distribution of $\mathbf{x}_T$ known: $q(\mathbf{x}_T \mid \mathbf{x}_0) \approx p(\mathbf{x}_T) = \mathcal{N}(\mathbf{0}, \mathbf{I})$



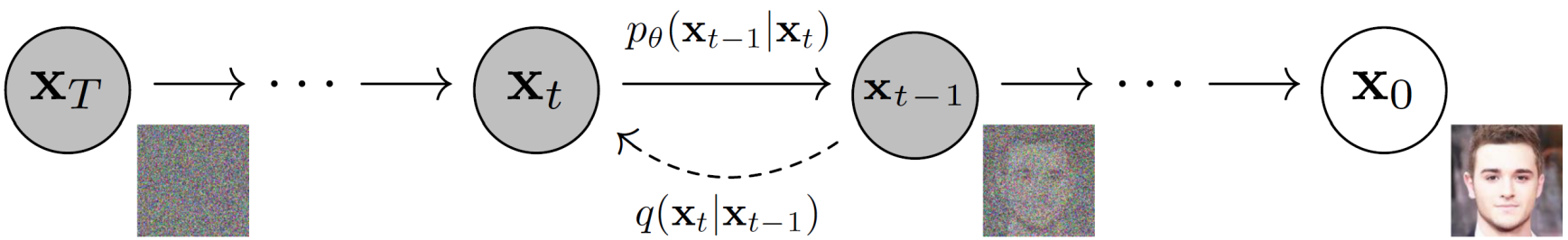
DIFFUSION MODELS AS MARKOVIAN HVAES

- Diffusion models are Markovian HVAEs with three restrictions:
  1. Latents have same dimensionality as data: $\mathbf{x}_0 := \mathbf{x}$, $\mathbf{x}_t := \mathbf{z}_t$
  2. The encoder is fixed: $q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$ gradually adds noise
  3. Distribution of $\mathbf{x}_T$ known: $q(\mathbf{x}_T \mid \mathbf{x}_0) \approx p(\mathbf{x}_T) = \mathcal{N}(\mathbf{0}, \mathbf{I})$
- Consequence: only the decoder $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ is learned



DIFFUSION MODELS: INTUITION

- Idea: gradually turn data into noise, and learn to reverse this process ▸ Ho et al. (2020)[2]
- **Forward process** q: fixed, adds a little noise in each step
- **Reverse process** p_θ: learned, removes a little noise in each step



FORWARD PROCESS

- The encoder is a fixed Markov chain without learnable parameters:

$$q(\mathbf{x}_{1:T} \mid \mathbf{x}_0) = \prod_{t=1}^{T} q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$$

- We start with a sample $\mathbf{x}_0$ and add Gaussian noise at each step:

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}\left(\mathbf{x}_t; \sqrt{\alpha_t}\,\mathbf{x}_{t-1}, (1-\alpha_t)\,\mathbf{I}\right)$$

- Equivalently: $\mathbf{x}_t = \sqrt{\alpha_t}\,\mathbf{x}_{t-1} + \sqrt{1-\alpha_t}\,\boldsymbol{\epsilon}_t$ with $\boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- **Noise schedule** $\alpha_1, \ldots, \alpha_T \in (0, 1)$: hyperparameters



CLOSED-FORM MARGINALS

- Unrolling the recursion:

$$
\begin{aligned}
\mathbf{x}_t &= \sqrt{\alpha_t}\,\mathbf{x}_{t-1} + \sqrt{1-\alpha_t}\,\boldsymbol{\epsilon}_t \\
&= \sqrt{\alpha_t}\left(\sqrt{\alpha_{t-1}}\,\mathbf{x}_{t-2} + \sqrt{1-\alpha_{t-1}}\,\boldsymbol{\epsilon}_{t-1}\right) + \sqrt{1-\alpha_t}\,\boldsymbol{\epsilon}_t \\
&= \sqrt{\alpha_t\alpha_{t-1}}\,\mathbf{x}_{t-2} + \sqrt{\alpha_t(1-\alpha_{t-1})}\,\boldsymbol{\epsilon}_{t-1} + \sqrt{1-\alpha_t}\,\boldsymbol{\epsilon}_t \\
&= \sqrt{\alpha_t\alpha_{t-1}}\,\mathbf{x}_{t-2} + \sqrt{1-\alpha_t\alpha_{t-1}}\,\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \\
&= \cdots = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}, \qquad \bar{\alpha}_t := \prod_{s=1}^{t} \alpha_s
\end{aligned}
$$

- Hence $q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}\left(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0, (1-\bar{\alpha}_t)\,\mathbf{I}\right)$
- We can jump to any t directly, without simulating the chain
- $\bar{\alpha}_t$ decreases from ≈ 1 to ≈ 0, so $q(\mathbf{x}_T \mid \mathbf{x}_0) \approx \mathcal{N}(\mathbf{0}, \mathbf{I})$: $\mathbf{x}_T$ carries (almost) no information about $\mathbf{x}_0$

BACKWARD PROCESS

- Generative model (decoder):

$$p_\theta(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^{T} p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t), \qquad p(\mathbf{x}_T) = \mathcal{N}(\mathbf{0}, \mathbf{I})$$

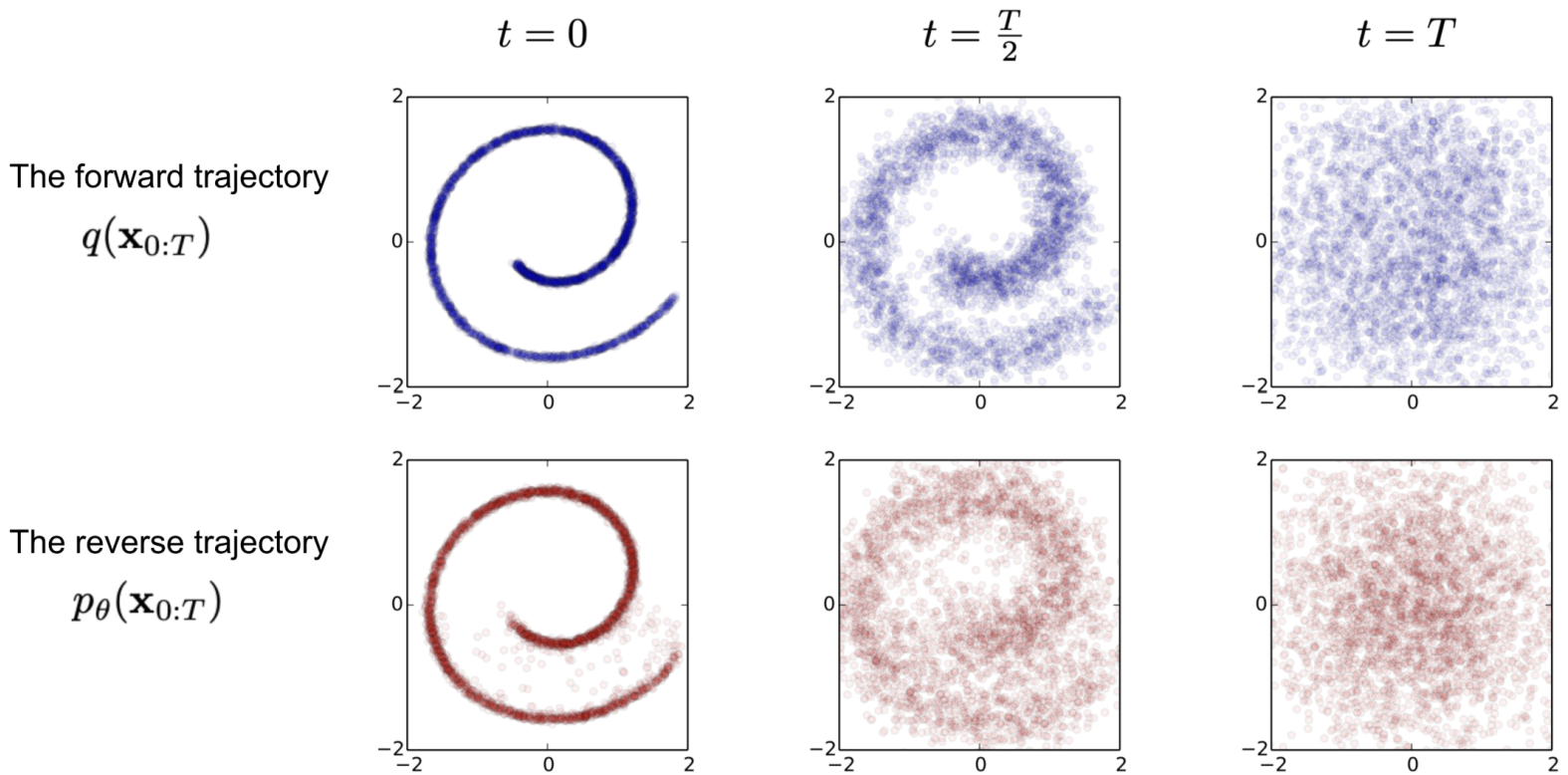
- We do not know this process, but we will try to learn it from data
- Each step is Gaussian (mean predicted by a neural network):

$$p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N}\big(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I}\big)$$

- To generate from this model, we sample $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, then sample $\mathbf{x}_{t-1} \sim p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ for $t = T, \ldots, 1$
- The ideal reverse step $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ is intractable but approximately Gaussian for small noise steps

DIFFUSION MODEL TRAINING INTUITION

- Our goal is to learn the backward process $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ from the forward process $q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$
- We will use the KL divergence for that



ELBO FOR DIFFUSION MODELS (I)

- Markovian HVAE ELBO with $\mathbf{z}_t = \mathbf{x}_t$ and a fixed encoder, so it only depends on θ ($\mathbb{E}_q$ short for $\mathbb{E}_{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}$):

$$\text{ELBO}(\theta; \mathbf{x}_0) = \mathbb{E}_q \left[\log \frac{p(\mathbf{x}_T)\, p_\theta(\mathbf{x}_0 \mid \mathbf{x}_1) \prod_{t=2}^{T} p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)}{q(\mathbf{x}_1 \mid \mathbf{x}_0) \prod_{t=2}^{T} q(\mathbf{x}_t \mid \mathbf{x}_{t-1})} \right]$$

- Key trick: rewrite the encoder steps using $\mathbf{x}_0$ since $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ is intractable, but $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$ is not!

ELBO FOR DIFFUSION MODELS (II)

- Apply Markov property, then Bayes' rule (conditioned on $\mathbf{x}_0$):

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = q(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{x}_0) = \frac{q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)\, q(\mathbf{x}_t \mid \mathbf{x}_0)}{q(\mathbf{x}_{t-1} \mid \mathbf{x}_0)}$$

- The product of ratios $\prod_{t=2}^{T} \frac{q(\mathbf{x}_t|\mathbf{x}_0)}{q(\mathbf{x}_{t-1}|\mathbf{x}_0)}$ telescopes:

$$\frac{q(\mathbf{x}_2 \mid \mathbf{x}_0)}{q(\mathbf{x}_1 \mid \mathbf{x}_0)} \cdot \frac{q(\mathbf{x}_3 \mid \mathbf{x}_0)}{q(\mathbf{x}_2 \mid \mathbf{x}_0)} \cdots \frac{q(\mathbf{x}_T \mid \mathbf{x}_0)}{q(\mathbf{x}_{T-1} \mid \mathbf{x}_0)} = \frac{q(\mathbf{x}_T \mid \mathbf{x}_0)}{q(\mathbf{x}_1 \mid \mathbf{x}_0)}$$

- Hence the denominator of the ELBO becomes:

$$q(\mathbf{x}_1 \mid \mathbf{x}_0) \prod_{t=2}^{T} q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = q(\mathbf{x}_T \mid \mathbf{x}_0) \prod_{t=2}^{T} q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$$

- Interpretation: $q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)$ factorized in reverse order

ELBO FOR DIFFUSION MODELS (III)

- Substituting the reverse factorization and splitting the logarithm:

$$
\begin{aligned}
\text{ELBO} &= \mathbb{E}_q \left[\log \frac{p(\mathbf{x}_T)\, p_\theta(\mathbf{x}_0 \mid \mathbf{x}_1) \prod_{t=2}^{T} p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)}{q(\mathbf{x}_T \mid \mathbf{x}_0) \prod_{t=2}^{T} q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)} \right] \\
&= \mathbb{E}_q \left[\log p_\theta(\mathbf{x}_0 \mid \mathbf{x}_1) + \log \frac{p(\mathbf{x}_T)}{q(\mathbf{x}_T \mid \mathbf{x}_0)} + \sum_{t=2}^{T} \log \frac{p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)}{q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)} \right] \\
&= \underbrace{\mathbb{E}_{q(\mathbf{x}_1 \mid \mathbf{x}_0)} \left[\log p_\theta(\mathbf{x}_0 \mid \mathbf{x}_1) \right]}_{\text{reconstruction term}} - \underbrace{\text{KL}\left(q(\mathbf{x}_T \mid \mathbf{x}_0) \,\|\, p(\mathbf{x}_T) \right)}_{\text{prior matching term}} \\
&\quad - \sum_{t=2}^{T} \underbrace{\mathbb{E}_{q(\mathbf{x}_t \mid \mathbf{x}_0)} \left[\text{KL}\left(q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \,\|\, p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) \right) \right]}_{\text{denoising matching term}}
\end{aligned}
$$

- NB: expectations reduce to the latents each term involves

ELBO INTERPRETATION

- **Reconstruction term**: recover $\mathbf{x}_0$ from the slightly noisy $\mathbf{x}_1$
- **Prior matching term**: no learnable parameters and ≈ 0 (why?)
- **Denoising matching term**: each learned reverse step $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ should match the "ground-truth" reverse step $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$, which knows $\mathbf{x}_0$
- The derivation only used the Markov property and Bayes' rule, so it holds for *any* forward process

TRACTABLE REVERSE STEP

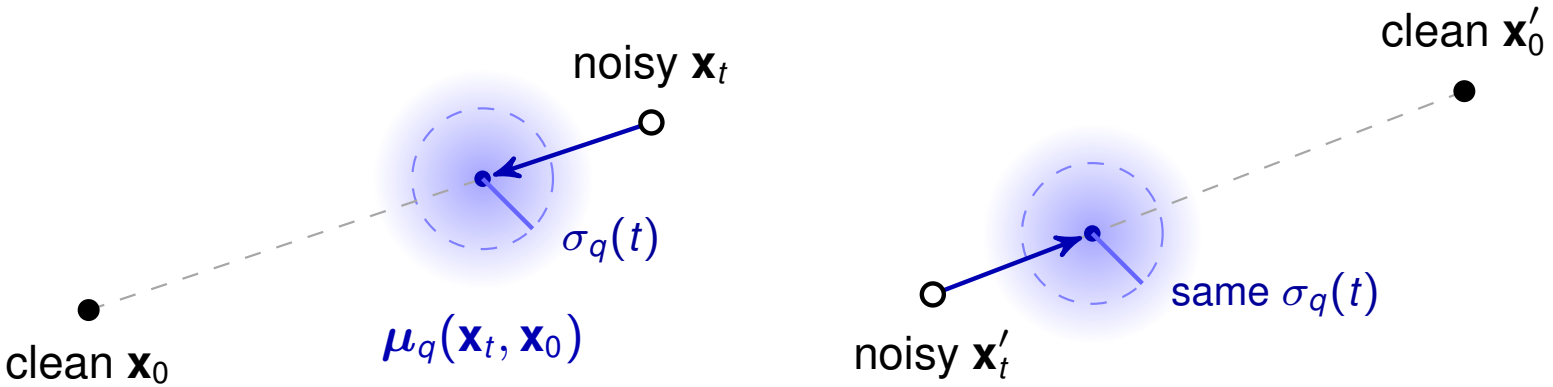
- Bayes' rule, where all three terms are Gaussians we know:

$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) = \frac{q(\mathbf{x}_t \mid \mathbf{x}_{t-1})\, q(\mathbf{x}_{t-1} \mid \mathbf{x}_0)}{q(\mathbf{x}_t \mid \mathbf{x}_0)}$$
$$= \frac{\mathcal{N}\left(\mathbf{x}_t; \sqrt{\alpha_t}\,\mathbf{x}_{t-1}, (1-\alpha_t)\,\mathbf{I}\right) \mathcal{N}\left(\mathbf{x}_{t-1}; \sqrt{\bar{\alpha}_{t-1}}\,\mathbf{x}_0, (1-\bar{\alpha}_{t-1})\,\mathbf{I}\right)}{\mathcal{N}\left(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0, (1-\bar{\alpha}_t)\,\mathbf{I}\right)}$$

- This can be rewritten as another Gaussian:

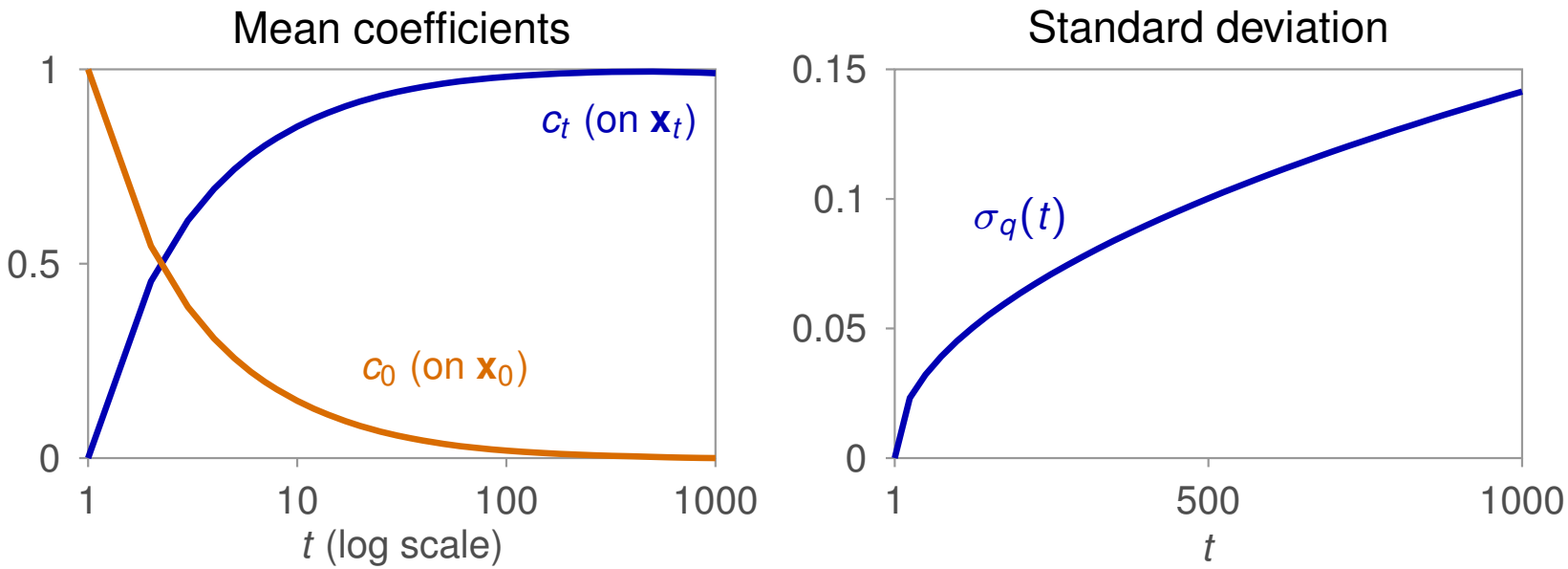
$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}\left(\mathbf{x}_{t-1}; \boldsymbol{\mu}_q(\mathbf{x}_t, \mathbf{x}_0), \sigma_q^2(t)\,\mathbf{I}\right)$$
$$\boldsymbol{\mu}_q(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\alpha_t}\,(1-\bar{\alpha}_{t-1})\,\mathbf{x}_t + \sqrt{\bar{\alpha}_{t-1}}\,(1-\alpha_t)\,\mathbf{x}_0}{1-\bar{\alpha}_t}$$
$$\sigma_q^2(t) = \frac{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}$$

REVERSE STEP INTUITION



- Mean: $\boldsymbol{\mu}_q(\mathbf{x}_t, \mathbf{x}_0) = c_t\,\mathbf{x}_t + c_0\,\mathbf{x}_0$ with $c_t + c_0 \approx 1$, i.e., a step from the noisy $\mathbf{x}_t$ toward the clean $\mathbf{x}_0$ (small, except close to $t = 1$)
- Variance: $\sigma_q^2(t)$ only depends on the noise schedule, so all pairs $(\mathbf{x}_0, \mathbf{x}_t)$ at step t get the same spread

HOW THE REVERSE STEP CHANGES OVER TIME



- DDPM ▸ Ho et al. (2020)[2]: $T = 1000$, $1 - \alpha_t$ from 10^{-4} to 0.02
- $c_t \approx 1$ for almost all t: each reverse step stays close to $\mathbf{x}_t$
- c_0 is only large near $t = 1$ (where $\mu_q = \mathbf{x}_0$) and gives a small nudge toward $\mathbf{x}_0$ elsehere
- $\sigma_q(t)$ grows with t

INTERIM RECAP

- We model reverse steps as Gaussians:

$$p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N}\big(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I}\big)$$

- We want each learned reverse step $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ to match the "ground-truth" reverse step $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$
- We now have an exact formula for the "ground-truth" reverse step:

$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}\big(\mathbf{x}_{t-1}; \boldsymbol{\mu}_q(\mathbf{x}_t, \mathbf{x}_0), \sigma_q^2(t) \mathbf{I}\big)$$

- So why not simply set $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) := q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$?

INTERIM RECAP

- We model reverse steps as Gaussians:

$$p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N}\big(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I}\big)$$

- We want each learned reverse step $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ to match the "ground-truth" reverse step $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$
- We now have an exact formula for the "ground-truth" reverse step:

$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}\big(\mathbf{x}_{t-1}; \boldsymbol{\mu}_q(\mathbf{x}_t, \mathbf{x}_0), \sigma_q^2(t) \mathbf{I}\big)$$

- So why not simply set $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) := q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$?
- The variance is easy: $\sigma_t^2 := \sigma_q^2(t)$
- The mean is the problem: $\boldsymbol{\mu}_q$ needs $\mathbf{x}_0$, but p_θ only sees $\mathbf{x}_t$

LEARNING TO RECOVER THE ORIGINAL DATA

- Solution: we plug in a neural network prediction $\hat{\mathbf{x}}_\theta(\mathbf{x}_t, t) \approx \mathbf{x}_0$:

$$p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N}\big(\mathbf{x}_{t-1}; \boldsymbol{\mu}_q(\mathbf{x}_t, \hat{\mathbf{x}}_\theta(\mathbf{x}_t, t)), \sigma_q^2(t)\,\mathbf{I}\big)$$

- KL between Gaussians with equal covariance:

$$\begin{aligned}
&\mathrm{KL}\left(q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \,\|\, p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)\right) \\
&\quad = \frac{1}{2\sigma_q^2(t)} \left\| \boldsymbol{\mu}_q(\mathbf{x}_t, \mathbf{x}_0) - \boldsymbol{\mu}_q(\mathbf{x}_t, \hat{\mathbf{x}}_\theta(\mathbf{x}_t, t)) \right\|^2 \\
&\quad = \underbrace{\frac{c_0^2}{2\sigma_q^2(t)}}_{=:\, w_t} \left\| \hat{\mathbf{x}}_\theta(\mathbf{x}_t, t) - \mathbf{x}_0 \right\|^2
\end{aligned}$$

- w_t is large at low noise (large c_0) and tiny at high noise
- **A diffusion model is a denoiser that learns to recover the original data at all noise levels!**

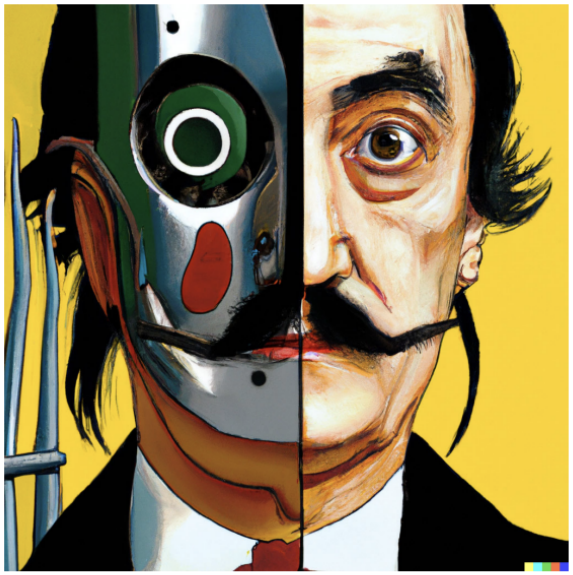
TRAINING

- Objective (the prior matching term is constant):

$$\min_{\theta} \sum_{t=2}^{T} \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} \left[w_t \left\| \hat{\mathbf{x}}_\theta(\mathbf{x}_t, t) - \mathbf{x}_0 \right\|^2 \right]$$

- Since w_t is largest for small t, the loss implicitly focuses low-noise reconstructions
- The reconstruction term ($t = 1$) can be handled analogously
- Repeat until convergence (Monte Carlo estimate):
  1. Sample a training example $\mathbf{x}_0$
  2. Sample a noise level $t \sim \mathcal{U}\{2, \ldots, T\}$
  3. Sample $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and set $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\, \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\, \boldsymbol{\epsilon}$
  4. Take a gradient step on $w_t \left\| \hat{\mathbf{x}}_\theta(\mathbf{x}_t, t) - \mathbf{x}_0 \right\|^2$
- One network $\hat{\mathbf{x}}_\theta$ is shared across all noise levels

DIFFUSION MODELS IN ACTION



vibrant portrait painting of Salvador Dalí with a robotic half face



a shiba inu wearing a beret and black turtleneck



a close up of a handpalm with leaves growing from it

- Diffusion models power most image generators (e.g., DALLE)
- But images are continuous, while text is discrete: how do we add “noise” to text?

OUTLINE

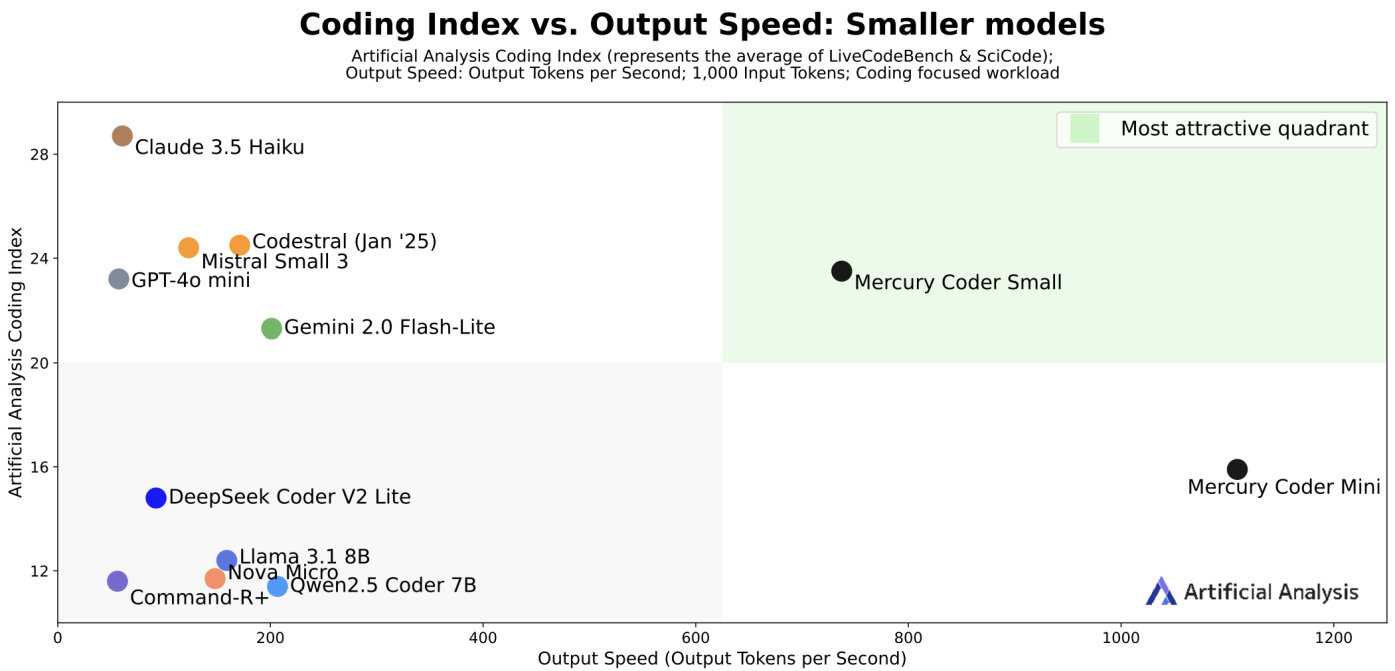
Deep Latent Variable Models

Diffusion Models

Masked Diffusion LLMs

WHY DIFFUSION FOR TEXT?

Autoregression	Diffusion
✗ Sequential generation	✓ Parallel (fast) generation
✗ Fixed number of steps	✓ Flexible number of steps
✗ Left-to-right context	✓ Bidirectional context

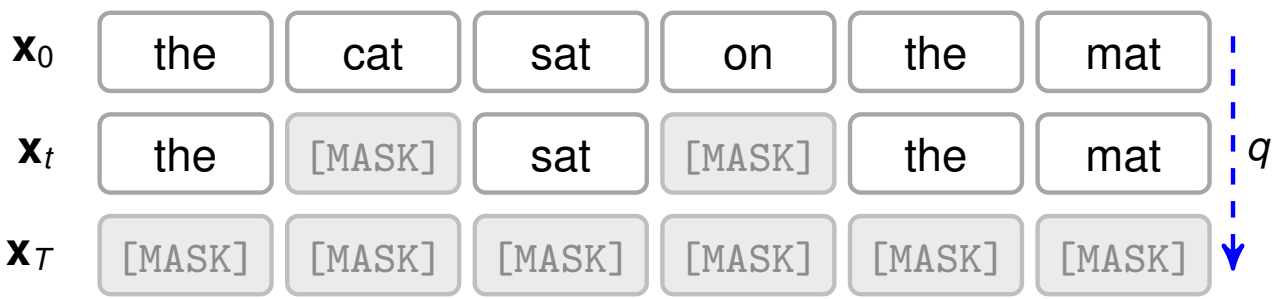


MASKING AS THE FORWARD PROCESS

- Tokens are one-hot vectors over $\mathcal{V} \cup \{\texttt{[MASK]}\}$
- We start with a clean sequence $\mathbf{x}_0 = (\mathbf{x}_0^1, \ldots, \mathbf{x}_0^L)$
- Tokens are masked independently ($\mathbf{m}$ denotes vector of `[MASK]`):

$$q(\mathbf{x}_t^\ell \mid \mathbf{x}_0^\ell) = \mathsf{Cat}\left(\mathbf{x}_t^\ell;\ \bar{\alpha}_t\, \mathbf{x}_0^\ell + (1 - \bar{\alpha}_t)\, \mathbf{m}\right)$$

- Keep the token with probability $\bar{\alpha}_t$, otherwise mask it
- Masked tokens stay masked (**absorbing state**)
- $\bar{\alpha}_0 = 1$ and $\bar{\alpha}_T = 0$, so $\mathbf{x}_T$ is all `[MASK]`



TRACTABLE REVERSE STEP

- Unmasked tokens stay unmasked: if $\mathbf{x}_t^\ell \neq \mathbf{m}$, then $\mathbf{x}_{t-1}^\ell = \mathbf{x}_t^\ell$
- Masked tokens:

$$q(\mathbf{x}_{t-1}^\ell \mid \mathbf{x}_t^\ell = \mathbf{m}, \mathbf{x}_0^\ell) = \mathsf{Cat}\left(\mathbf{x}_{t-1}^\ell;\ w_t\,\mathbf{x}_0^\ell + (1 - w_t)\,\mathbf{m}\right)$$

- A masked token is unmasked to $\mathbf{x}_0^\ell$ with probability $w_t := \frac{\bar{\alpha}_{t-1} - \bar{\alpha}_t}{1 - \bar{\alpha}_t}$ and stays masked otherwise
- Same structure as in the Gaussian case: a weighted combination of $\mathbf{x}_t$ ($= \mathbf{m}$) and $\mathbf{x}_0$

LEARNING TO RECOVER THE ORIGINAL TOKENS

- As before, $p_\theta(\mathbf{x}_{t-1}^\ell \mid \mathbf{x}_t)$ does not have access to $\mathbf{x}_0$, so we predict it with a neural network, $\hat{\mathbf{x}}_\theta^\ell(\mathbf{x}_t)$, and compute:

$$p_\theta(\mathbf{x}_{t-1}^\ell \mid \mathbf{x}_t) = \mathrm{Cat}\left(\mathbf{x}_{t-1}^\ell;\ w_t\, \hat{\mathbf{x}}_\theta^\ell(\mathbf{x}_t) + (1 - w_t)\, \mathbf{m}\right) \quad \text{if } \mathbf{x}_t^\ell = \mathbf{m}$$

- Unmasked positions are copied, so their KL is 0
- Masked positions: KL simplifies as

$$\mathrm{KL}\left(q(\mathbf{x}_{t-1}^\ell \mid \mathbf{x}_t^\ell, \mathbf{x}_0^\ell) \,\|\, p_\theta(\mathbf{x}_{t-1}^\ell \mid \mathbf{x}_t)\right) = -\, w_t \log p_\theta(\mathbf{x}_0^\ell \mid \mathbf{x}_t)$$

- $p_\theta(\mathbf{x}_0^\ell \mid \mathbf{x}_t)$ is the probability $\hat{\mathbf{x}}_\theta^\ell(\mathbf{x}_t)$ assigns to the correct token
- Weighted cross-entropy (cf. $w_t \|\hat{\mathbf{x}}_\theta - \mathbf{x}_0\|^2$ in the Gaussian case)

TRAINING OBJECTIVE

- Prior matching term is 0 (why?)
- Negative ELBO ▸ Sahoo et al. (2024)[6]:

$$\mathcal{L}(\theta) = \sum_{t=1}^{T} \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} \left[w_t \sum_{\ell:\, \mathbf{x}_t^\ell = \mathbf{m}} - \log p_\theta(\mathbf{x}_0^\ell \mid \mathbf{x}_t) \right]$$

- What has happened to the reconstruction term?
- Linear schedule $\bar{\alpha}_t = 1 - t/T$: masking rate $r_t = t/T$ and $w_t = 1/t$, so

$$\mathcal{L}(\theta) = \mathbb{E}_{t\sim\mathcal{U}\{1,\ldots,T\}} \, \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} \left[\frac{1}{r_t} \sum_{\ell:\, \mathbf{x}_t^\ell = \mathbf{m}} - \log p_\theta(\mathbf{x}_0^\ell \mid \mathbf{x}_t) \right]$$

- Again, lightly masked inputs (low noise) get the largest weight

SURPRISE: MDLM IS (ALMOST) BERT!

- BERT: mask 15% of the tokens and predict them ▸ Devlin et al. (2019)[1]

$$\mathcal{L}_{\text{BERT}}(\theta) = \mathbb{E}_{\mathbf{x}_t:\ 15\%\ \text{masked}} \Bigg[\sum_{\ell:\, \mathbf{x}_t^\ell = \mathbf{m}} -\log p_\theta(\mathbf{x}_0^\ell \mid \mathbf{x}_t) \Bigg]$$

SURPRISE: MDLM IS (ALMOST) BERT!

- BERT: mask 15% of the tokens and predict them ▸ Devlin et al. (2019)[1]

$$\mathcal{L}_{\text{BERT}}(\theta) = \mathbb{E}_{\mathbf{x}_t:\ 15\%\ \text{masked}} \left[\sum_{\ell:\, \mathbf{x}_t^\ell = \mathbf{m}} -\log p_\theta(\mathbf{x}_0^\ell \mid \mathbf{x}_t) \right]$$

- MDLM: same loss, with a random masking rate and a weight

$$\mathcal{L}_{\text{MDLM}}(\theta) = \mathbb{E}_{r \sim \mathcal{U}(0,1]}\, \mathbb{E}_{\mathbf{x}_t:\ r\ \text{masked}} \left[\frac{1}{r} \sum_{\ell:\, \mathbf{x}_t^\ell = \mathbf{m}} -\log p_\theta(\mathbf{x}_0^\ell \mid \mathbf{x}_t) \right]$$

SURPRISE: MDLM IS (ALMOST) BERT!

- BERT: mask 15% of the tokens and predict them ▸ Devlin et al. (2019)[1]

$$\mathcal{L}_{\text{BERT}}(\theta) = \mathbb{E}_{\mathbf{x}_t:\ 15\%\ \text{masked}} \left[\sum_{\ell:\, \mathbf{x}_t^\ell = \mathbf{m}} -\log p_\theta(\mathbf{x}_0^\ell \mid \mathbf{x}_t) \right]$$

- MDLM: same loss, with a random masking rate and a weight

$$\mathcal{L}_{\text{MDLM}}(\theta) = \mathbb{E}_{r \sim \mathcal{U}(0,1]}\, \mathbb{E}_{\mathbf{x}_t:\ r\ \text{masked}} \left[\frac{1}{r} \sum_{\ell:\, \mathbf{x}_t^\ell = \mathbf{m}} -\log p_\theta(\mathbf{x}_0^\ell \mid \mathbf{x}_t) \right]$$

- These two changes turn the loss into an upper bound on $-\log p_\theta(\mathbf{x}_0)$, making MDLMs proper generative models
- BERT never learns to predict from fully masked inputs, so it cannot generate from scratch

SAMPLING

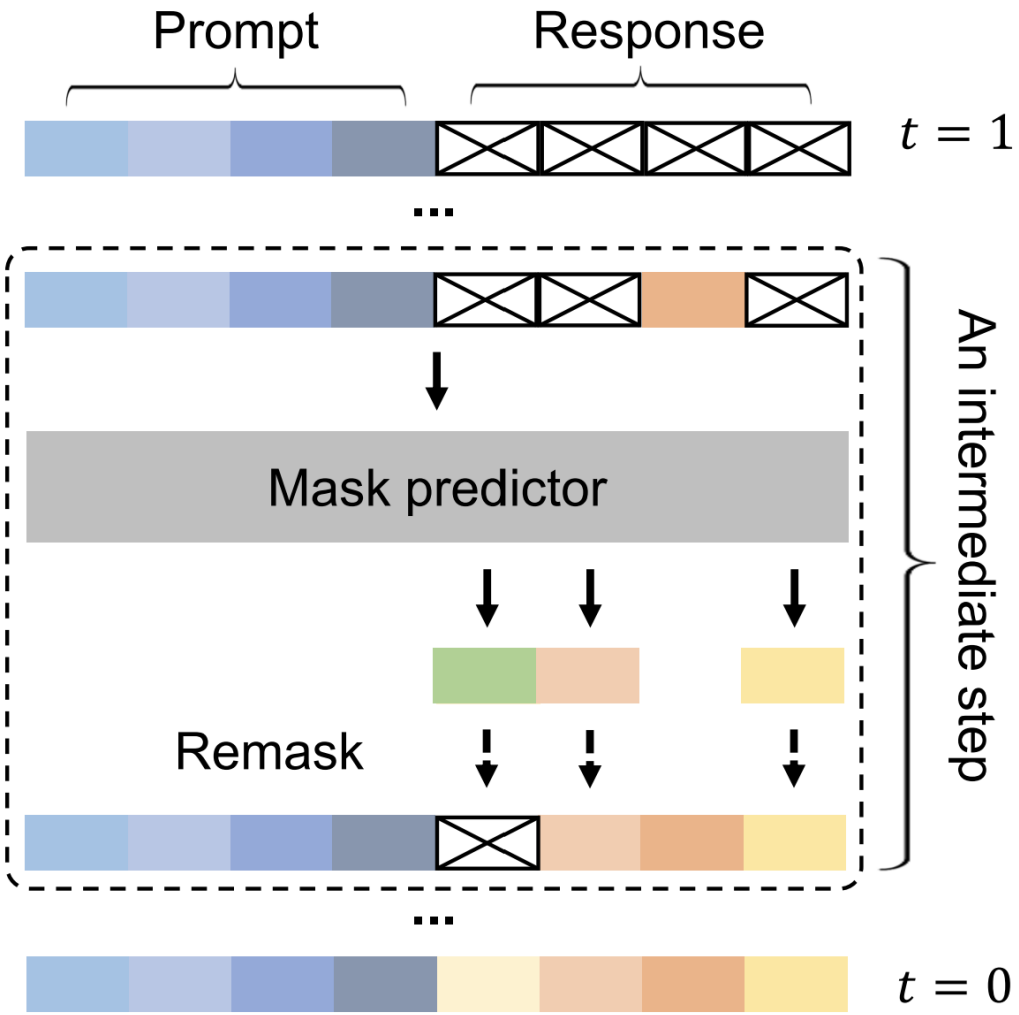
- Start from $\mathbf{x}_T$: all `[MASK]` (output length fixed in advance)
- For $t = T, \ldots, 1$: predict $\hat{\mathbf{x}}_\theta^\ell(\mathbf{x}_t)$ for all masked positions in parallel, unmask each with probability w_t, and sample its token from $\hat{\mathbf{x}}_\theta^\ell(\mathbf{x}_t)$
- Unmasked tokens never change; in practice, the most confident predictions are often unmasked first ▸ Nie et al. (2025)[4]
- Caveat: tokens unmasked in the same step are sampled independently (e.g., *San York*)

ITERATIVE REFINEMENT

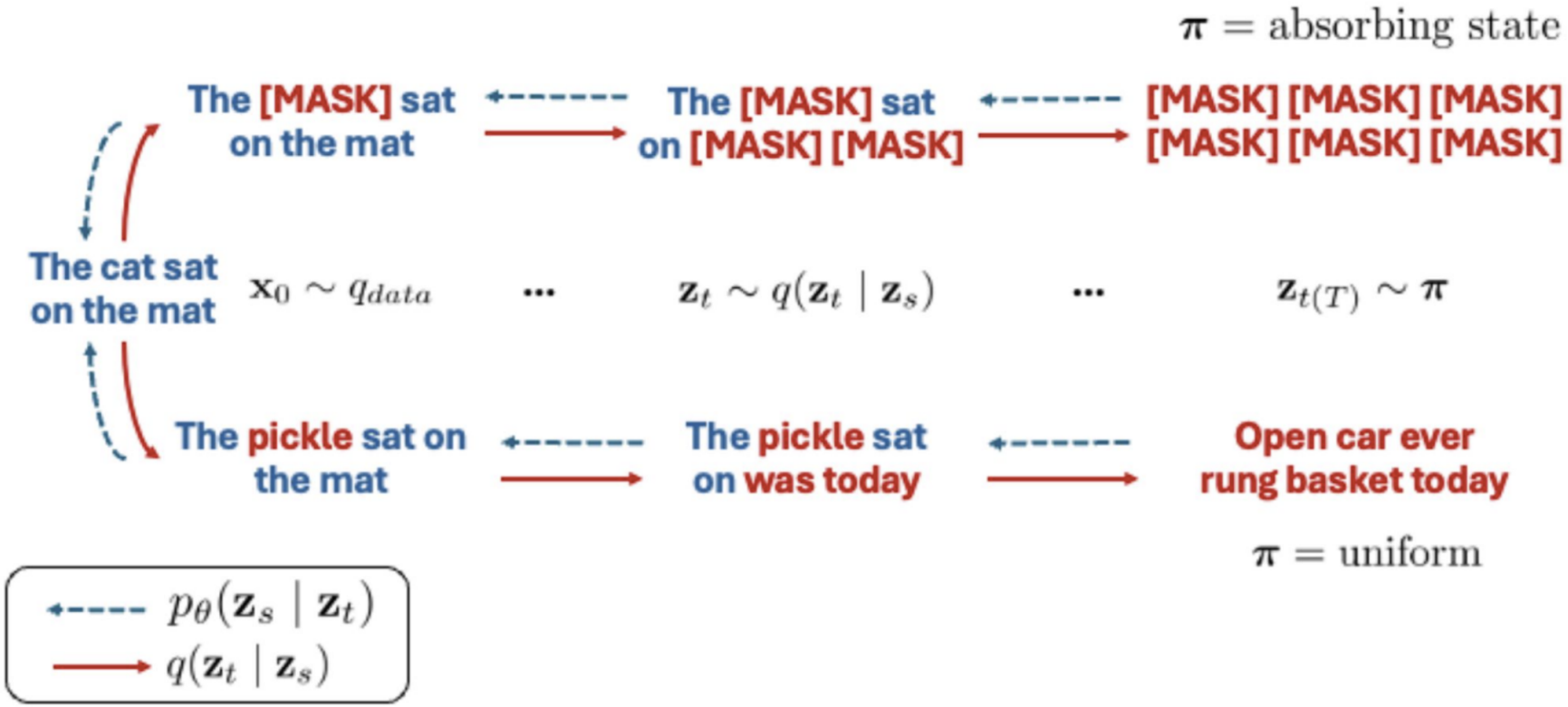


Model produces "sell" because it's a valid completion, but later corrects itself

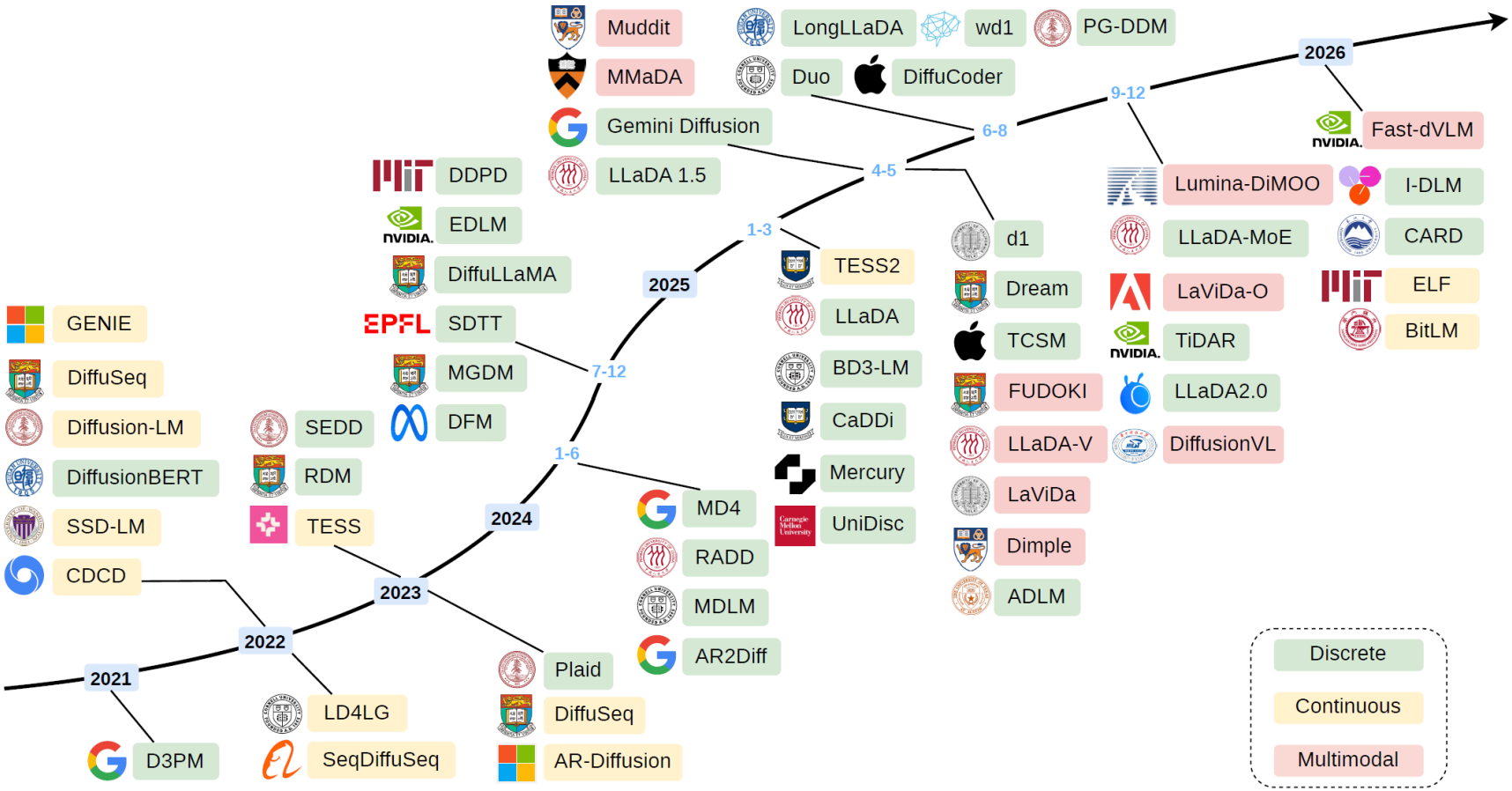
PROMPTING WITH MDLMS



ALTERNATIVES TO MASKED DIFFUSION



VERY ACTIVE RESEARCH AREA



REFERENCES

[1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4171–4186, 2019.

[2] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Conference on Neural Information Processing Systems*, 2020.

[3] Diederik P. Kingma and Max Welling. Auto-encoding variational Bayes. In *International Conference on Learning Representations*, 2014.

[4] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. *arXiv preprint arXiv:2502.09992*, 2025.

[5] Danilo Jimenez Rezende, Shakir Mohamed, and Daan Wierstra. Stochastic backpropagation and approximate inference in deep generative models. In *Proceedings of the 31st International Conference on Machine Learning*, pages 1278–1286, 2014.

[6] Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T. Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[7] Arash Vahdat and Jan Kautz. NVAE: A deep hierarchical variational autoencoder. In *Advances in Neural Information Processing Systems*, volume 33, 2020.